

考虑因果预测与日内调账的微网电力调控策略

摘要

针对含光伏、负荷与储能的微网在日前购电、日内合同调整和变动电价下的协同调度问题,本文建立一套严格遵守信息发布时间的“合同层滚动线性规划+因果贪心反馈”框架。首先将附件中以区间终点标记的10分钟数据统一映射为自然时段,把功率换算为每槽电量,并以母线平衡、储能效率、功率上限和荷电状态约束构成统一物理层。问题一采用循环日连续线性规划,以购电费用为主目标、储能吞吐量为目标,得到典型日购电量59482.6982kWh、费用35126.9489元,且荷电状态由6000kWh出发并回到同一水平。

对于问题二,本文只使用决策时刻之前的历史数据:以90日以内相似日得到负荷和光伏点预测,以最近60个已完成日期的原始联合残差形成“联合残差特征提取后的边际分位安全轨迹”,再求解单条确定性线性规划。正式风险模式为未聚类原始残差,聚类只作为固定参数的压缩对照;正式统计覆盖2025年2月至12月共334日,1月仅用于历史启动和储能状态连续推进。固定电价下,无日内调账策略的现金费用为18446381.69元,应急购电为499461.62kWh。相似日负荷预测的MAE和RMSE分别为722.57、904.09kW;历史90%经验区间的实际覆盖率低于名义水平,因此本文不把它解释为概率保证。

对于问题三,在0、6、12、18时接收官方光伏预测,并以最近60个已完成日期的同发布时刻误差作因果偏差修正,只重算尚未执行的合同。该正式策略费用为16222449.30元、应急购电为100703.52kWh,较问题二节约2223932.39元;这一差异同时包含信息改善、合同调整权和执行路径变化,不能全部归因于更新次数。为隔离官方信息价值,固定0时版本并对 $S \subseteq \{6, 12, 18\}$ 的8组可读发布时间做完整消融:各组均保留0、6、12、18时重优化,只屏蔽指定版本及其历史校准残差。全集相对冻结0时节省381887.28元,应急购电减少28433.21kWh,其中12时信息的Shapley贡献最大。相同0时信息下,2小时合成更新相对6小时方案节省628384.73元;替代结算下仅节省9074.01元。

对于问题四,未来电价预测只采用过去30日同槽中位数及当日已实现偏差,实际电价仅用于结算。无调账与有调账方案的现金费用分别为1892.44万元和1667.05万元,后者应急购电为100932.41kWh。变价条件下2小时方案相对6小时方案节省625985.25元;若每次额外更新成本高于约234.28元,则该增频不再具有净经济优势。通过54组基线与单因素实验、逐槽能量平衡、跨日储能连续性、规划一执行偏差和未来信息扰动检验,验证了算法的数值可行性与因果性。本文方法适用于题设数据和结算假设,不等同于完整随机模型预测控制,也不提供样本外概率保证。

关键词: 微网调度; 因果预测; 滚动线性规划; 经验分位数; 储能; 合同调账

1 问题重述与分析

1.1 问题背景与任务拆解

微网由光伏电源、用电负荷、储能装置以及与外部电网相连的购电通道构成。光伏和负荷均具有显著的日内波动，且真实值在决策后才逐步显现；储能能够在时段之间转移电量，却受到容量、功率和往返损耗限制。若日前合同偏低，实时缺口必须以高倍率应急电价补齐；若合同偏高，多余合同电量又可能无法获得完全退款。因此，本题不是单纯的峰谷套利，而是预测误差、合同规则、储能状态和信息更新时间共同作用的序贯决策问题。

已有研究通常将微网不确定性表示为场景、随机过程或滚动预测控制问题 [2, 1]。两层随机模型预测控制可同时处理较慢的计划层与较快的反馈层 [3]，日前市场和储能联合优化也常用随机规划刻画 [5]。滚动时域方法则在新信息到达时重求后续决策 [4]。本题数据量有限，且需要输出指定时段的可核验合同、电池和应急购电结果。为此，本文采用“因果预测-分位轨迹-线性规划-实际执行”的透明结构：不假定未来真实量已知，不把复杂场景树直接塞入控制器，而是从历史联合残差提取一条可解释的保守轨迹。

四个问题形成逐层递进关系：

- (1) 问题一给出典型日负荷、光伏和固定电价，要求在日末恢复初始荷电状态。它用于确定物理约束、时间口径和确定性调度基线。
- (2) 问题二将负荷和光伏扩展至全年，但不允许日内调整购电合同。核心是用历史构造次日预测和不确定性余量，并让储能状态跨日连续。
- (3) 问题三在 0、6、12、18 时获得官方光伏预报，可对尚未执行的合同调账。核心是区分“当时已知信息”与“事后真实值”，并按题设规则只结算最终有效合同。
- (4) 问题四把外部电网电价改为随时间变化，要求分别给出无调账和有调账策略。核心是对未来价格也实行因果预测，实际价格只在结算时使用。

从决策目标看，本题至少包含三组相互牵制的量。第一组是合同费用与应急费用：增加日前合同会降低高价应急缺口，却可能产生更多闲置合同；第二组是供电可靠性与光伏消纳：保守净负荷有助于保障负荷，却可能挤占光伏在中午的消纳空间；第三组是当前收益与未来储能价值：过早放电会降低当前槽购电，但会增加后续高价槽的缺口。由此，本文不把单一电量或单一费用作为所有问题的评价标准，而是把现金费用作为优化目标，同时报告应急量、闲置合同、弃光和 SOC 末态，并用终端价值协调短期与后续时段。

问题间比较也需要统一合同。问题二与问题三使用相同固定电价和物理设备，问题四的两个方案使用相同变动电价序列；所有策略从 1 月 1 日同一 SOC 出发，2 月至 12 月采用完全相同的统计日期。只有在这种共同基础上，费用差才具有可比性。另一方面，问题二到问题三不仅增加更新事件，也更换了光伏信息源，因此本文另设“相同 0 时信息”的 0/12/6/2 小时实验来分析更新集合；这一步避免把预报源变化误计为纯频率收益。

1.2 信息集与评价口径

令日期为 d ，每天含 $T = 144$ 个 10 分钟时槽，槽序号为 $t = 0, 1, \dots, 143$ 。在日期 d 的事件时刻 h 做决策时，可用信息集记为 $\mathcal{F}_{d,h}$ 。它包括：所有早于 d 的完整日期数据、当天 h 时之前已经实现的负荷与光伏、截至 h 时已经发布的官方预报，以及变价情形下已经实现的价格。任何 h 时之后的真实负荷、真实光伏和真实价格都不属于 $\mathcal{F}_{d,h}$ 。

为避免首月没有历史而导致模型断裂，计算从 1 月 1 日起推进；历史不足时用附件典型日收缩估计。储能状态从 1 月 1 日连续传递到 12 月 31 日，不在日界处重置。题目要求的结果统计期为 2 月 1 日至 12 月 31 日，共 334 日。因此，文中“年度费用”若无特别说明，均指这 334 日的正式统计，而原始年度负荷、光伏和价格分布图可使用 365 日数据。

评价指标包括现金费用、应急购电量、闲置合同电量、弃光量、储能末态和逐槽物理残差。预测部分报告 MAE、RMSE、MAPE 及经验区间覆盖率，但 MAPE 仅在真实光伏大于 100kW 的样本上计算；若没有有效样本则记为缺失，而不是零误差。时间序列不是相互独立的重复实验，本文不使用未经验证的独立同分布显著性检验。

1.3 总体建模流程

模型首先将输入功率乘以 $\Delta t = 1/6\text{h}$ 得到时槽电量；随后基于 $\mathcal{F}_{d,h}$ 生成负荷、光伏和必要时的价格预测；再由历史联合残差提取边际分位安全轨迹，求解满足储能约束的线性规划；最后用真实量在当前槽执行“余电充电、缺电放电、剩余缺口应急购电”的因果策略。新预报到达时，仅对尚未执行的合同段重复上述过程。典型日输入结构见图1。

这一流程保留了滚动优化的反馈结构，但与完整随机 MPC 不同：控制器输入的是压缩后的确定性分位轨迹，而非带联合概率的完整场景树。这样可以直接追溯每一项输出，同时也意味着经验区间和分位参数只反映本样本中的风险暴露，不能被解释为严格的机会约束保证。

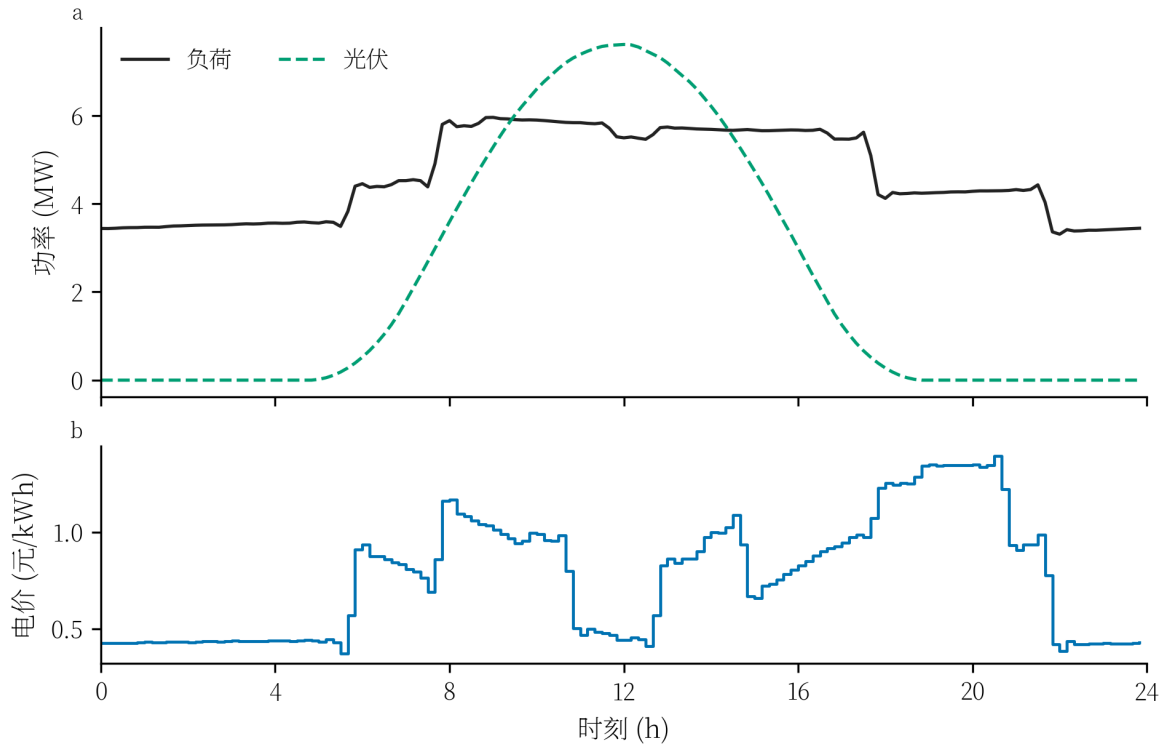


图 1 典型日负荷、光伏与固定电价的日内结构。负荷和光伏峰值并不完全同步，电价阶梯又改变了储能转移能量的价值。

2 模型假设、时间口径与符号

2.1 基本假设

在不改变题面物理含义的前提下作如下假设：

- (1) 微网不能向外部电网反向售电。光伏可被负荷利用、用于充电或弃光；已生效合同中未被负荷或储能消纳的部分记为闲置合同电量。
- (2) 储能额定容量为 12000 kWh，荷电状态限定在 10% 至 90%，即 $1200 \leq S \leq 10800$ kWh；充、放电功率上限均为 5000 kW，单向效率均为 0.9。
- (3) 题设电价非负，不允许售电，允许弃光，也不存在吞吐补贴。连续线性规划以储能吞吐量为同费用解的次级筛选量；第3.2节说明这些条件足以选择不同时充放的最优解。
- (4) 问题一为循环典型日，日末荷电状态等于日初；问题二至四按全年顺序推进，不施加每日循环约束。规划末端价值只用于抑制短视放电，不从报告的现金费用中扣除。
- (5) 相似日、残差、偏差和价格预测均只能使用当前决策时刻以前可得的数据。官方光伏预报按发布时间进入信息集，跨年不可得的目标不参与误差计算。

(6) 题设未给出设备启停、老化、备用容量和配电网潮流限制，因此主目标只计合同及应急费用；不虚构题面之外的货币成本。

2.2 10 分钟时间映射

附件中的时刻标签从 00:10 持续到 24:00。本文将标签解释为自然 10 分钟区间的终点：00:10 对应 00:00–00:10，24:00 对应 23:50–24:00。设原始功率为 P_t ，则时槽电量为

$$E_t = P_t \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

题面 Excel 模板表头保持不变，数值按槽位置写入；论文中的指定时段、SOC 边界点和聚合量统一采用自然区间。该约定避免把 10 分钟电量误作整小时电量，也避免把 24:00 错误解释为下一日首槽。

2.3 规划层与执行层的两层物理模型

合同层滚动线性规划只在决策时刻使用可得预测量。令 $\tilde{L}_t, \tilde{V}_t$ 为规划所用的安全负荷与可用光伏， g_t^P 为规划合同， v_t^P, x_t^P, y_t^P 分别为规划光伏利用、充电和放电， S_t^P 为目标 SOC，则规划平衡和状态递推为

$$g_t^P + v_t^P + y_t^P = \tilde{L}_t + x_t^P, \quad (2)$$

$$S_{t+1}^P = S_t^P + \eta_c x_t^P - \frac{y_t^P}{\eta_d}. \quad (3)$$

对应约束为

$$0 \leq v_t^P \leq \tilde{V}_t, \quad g_t^P \geq 0, \quad (4)$$

$$0 \leq x_t^P \leq P_{\max} \Delta t, \quad 0 \leq y_t^P \leq P_{\max} \Delta t, \quad (5)$$

$$S_{\min} \leq S_t^P \leq S_{\max}. \quad (6)$$

问题三、四允许调账时，再增加非负的向上、向下调整变量 δ_t^+, δ_t^- ，使最终合同相对 0 时基准合同满足 $q_t - p_t = \delta_t^+ - \delta_t^-$ 。执行记账量 u_t, w_t 不进入上述 LP。

真实时槽到达后，因果贪心反馈读取真实负荷 L_t 、真实可用光伏 V_t 和真实 SOC。令 g_t 为该槽已生效合同， v_t^E, x_t^E, y_t^E 为执行量， w_t 为未消纳合同， u_t 为应急购电，则

$$g_t - w_t + v_t^E + y_t^E + u_t = L_t + x_t^E, \quad (7)$$

$$S_{t+1}^E = S_t^E + \eta_c x_t^E - \frac{y_t^E}{\eta_d}. \quad (8)$$

其中

$$0 \leq v_t^E \leq V_t, \quad 0 \leq w_t \leq g_t, \quad u_t \geq 0, \quad (9)$$

$$0 \leq x_t^E \leq P_{\max} \Delta t, \quad 0 \leq y_t^E \leq P_{\max} \Delta t, \quad (10)$$

$$S_{\min} \leq S_t^E \leq S_{\max}. \quad (11)$$

这里 u_t, w_t 只在真实量到达后由执行规则确定，是执行 / 结算记账量，不是合同层线性规划的决策变量。实际有效电网供能为 $g_t - w_t + u_t$ ，不能用合同量 g_t 代替。两层都取 $\eta_c = \eta_d = 0.9$ ，且 $P_{\max} \Delta t = 833.3333 \text{ kWh}$ 。

2.4 主要符号

全文主要符号、单位和角色见表1。功率只在输入和图中使用 kW，进入优化器后统一乘以时长成为 kWh；SOC 也是 kWh，因而各平衡式量纲一致。

表 1 主要符号及单位

符号	单位	含义与角色
d, t, h	–	日期、10 分钟时槽、预报或调账发布时刻
L_t, V_t	kWh	真实时槽负荷与可用光伏；带波浪号者为规划安全轨迹
g_t^P	kWh	规划合同决策；有调账时最终生效合同记为 q_t
$v_t^P, x_t^P, y_t^P, S_t^P$	kWh	LP 中的光伏利用、充电、放电与目标 SOC
$v_t^E, x_t^E, y_t^E, S_t^E$	kWh	因果执行层的光伏利用、充电、放电与真实 SOC
u_t, w_t	kWh	应急购电、闲置合同；仅执行 / 结算记账，不是 LP 变量
c_t	元/kWh	外部电网基准价格或实际结算价格
p_t, q_t	kWh	0 时基准合同、执行前最终有效合同
δ_t^+, δ_t^-	kWh	最终合同相对 0 时合同的向上、向下调整量
α	–	保守净负荷经验分位水平，事前固定为 0.8
$\mathcal{F}_{d,h}$	–	日期 d 在时刻 h 可获得的信息集

执行平衡式显式区分合同量 g_t 与实际合同供能 $g_t - w_t$ 。若省略 w_t ，当合同超过负荷、充电能力和光伏消纳需求时，模型会被迫虚构用电或错误允许外送；类似地，弃光量为 $V_t - v_t^E$ ，与闲置合同来自不同能源侧，不能合并。

对于全年策略，真实日末 SOC 是下一日的日初状态。设第 d 日末为 $S_{d,144}^E$ ，则

$$S_{d+1,0}^E = S_{d,144}^E. \quad (12)$$

因此 1 月虽不进入正式费用统计，仍作为因果预热期并把 SOC 连续传给 2 月 1 日。若每天重置到 6000 kWh，相当于在午夜无成本注入或抽走能量，会系统性扭曲年度费用。

3 问题一：典型日循环优化

3.1 确定性线性规划

问题一的负荷、光伏和电价均已知，且不允许应急购电。令购电价格为 c_t ，主目标为

$$\min C_1 = \sum_{t=0}^{143} c_t g_t^P, \quad (13)$$

并在式(2)–(6)基础上增加循环边界 $S_0^P = S_{144}^P = 6000$ 。问题一没有应急购电机理，无须在 LP 中引入执行记账量 u_t 。该模型是线性规划。由于同一最优费用下可能存在多种等价储能轨迹，先求得最小费用 C_1^* ，再在

$$C_1 \leq C_1^* + 10^{-8} \max\{1, |C_1^*|\} \quad (14)$$

的容差内最小化 $\sum_t (x_t^P + y_t^P)$ 。因此，报告的费用具有主目标最优性，而储能轨迹是低吞吐代表解，不宣称是唯一最优轨迹。

3.2 循环恒等式与充放电互斥

将式(3)在全天求和，并利用循环边界，可得

$$\eta_c \sum_t x_t^P = \frac{1}{\eta_d} \sum_t y_t^P, \quad \sum_t y_t^P = \eta_c \eta_d \sum_t x_t^P. \quad (15)$$

这说明放电总量小于充电总量并非能量不守恒，而是往返损耗的直接结果。本文没有增加 0-1 互斥变量：在电价非负、不可售电、允许弃光且没有吞吐补贴时，总能找到费用不增而吞吐不增的代表解；二阶段最小吞吐目标会从同费用解中排除无意义的同时充放。数值结果中 144 个时槽的同时充放量均为 0，因此连续 LP 已经给出物理可执行的互斥轨迹。

3.3 调度结果

求解得到总购电量 59482.698167 kWh，现金费用 35126.948941 元；全天充电量和放电量分别为 20740.661758 与 16799.936024 kWh，弃光为 0。初末 SOC 均为 6000 kWh，全天最小与最大 SOC 恰为 1200 和 10800 kWh。逐槽母线平衡最大绝对残差为 9.10×10^{-13} kWh。

图2显示储能在低价时段吸收外购电或光伏，在高价时段放电。由于单向效率为 0.9，只有足够大的价差才能覆盖往返损耗；约束边界处的平段说明容量而非价格判断成为局部瓶颈。日末恢复 6000 kWh 保证该典型日策略不通过透支期末电量来虚降费用。

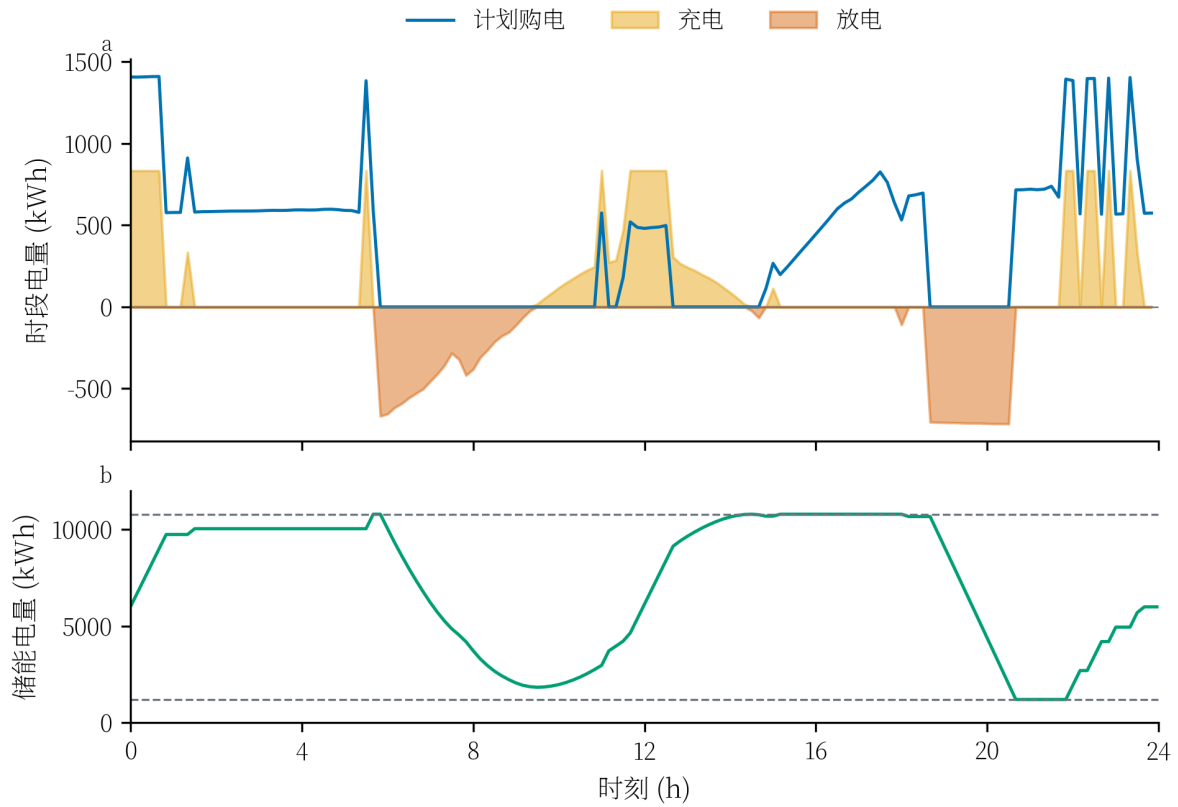


图 2 问题一最优购电、充放电与 SOC 轨迹。SOC 序列含 0 时和 24 时两个边界点。

题目指定的六个 10 分钟购电量列于表 2。其中 12:00、16:00 和 18:00 时起始槽需要购电，而 10:00、14:00 和 20:00 时起始槽购电为 0。这里每个数值对应单个 10 分钟槽，不是整小时电量。

表 2 问题一指定时段购电量

自然区间	10:00–10:10	12:00–12:10	14:00–14:10	16:00–16:10	18:00–18:10	20:00–20:10
购电量/kWh	0	480.41245	0	445.43165	531.89405	0

四小时聚合结果见表 3。充电量和放电量均由对应 24 个 10 分钟槽求和，SOC 则取自然 0 时和 24 时边界。

表 3 问题一各 4 小时储能电量

时段	充电/kWh	放电/kWh	时段	充电/kWh	放电/kWh
0–4 时	4500.000	0.000	12–16 时	5286.035	91.101
4–8 时	833.333	6365.838	16–20 时	0.000	5780.132
8–12 时	4787.960	1702.997	20–24 时	5333.333	2859.868
0 时 SOC: 6000 kWh			24 时 SOC: 6000 kWh		

若把全部标签整体解释为区间起点并循环平移 10 分钟，购电量几乎不变，而费用

变为 35126.848941 元，比主口径低约 0.10 元。这说明时间映射对总量稳健，但费用并非严格不变，故仍应统一自然区间定义。

4 问题二：无日内调账的因果分位轨迹优化

4.1 滚动相似日预测

问题二要求在每天开始前一次性确定全天合同，因而预测只能由过去日期生成。对目标日 d ，取最近 90 日内的历史日期集合 $\mathcal{H}_d$ 。对负荷，目标日与历史日星期属性相同时附加权重 2，否则权重 1；光伏不使用星期加权。时间衰减统一为

$$\omega_{dk} = a_{dk} \exp\left(-\frac{d-k}{60}\right), \quad k \in \mathcal{H}_d, \quad (16)$$

其中 $a_{dk} = 2$ 或 1。历史样本较少时，将加权历史均值向附件典型日收缩。以负荷为例，

$$\hat{L}_{d,t} = \gamma_d \frac{\sum_{k \in \mathcal{H}_d} \omega_{dk} L_{k,t}}{\sum_{k \in \mathcal{H}_d} \omega_{dk}} + (1 - \gamma_d) L_t^{\text{typ}}, \quad \gamma_d = \min\left(1, \frac{|\mathcal{H}_d|}{21}\right). \quad (17)$$

首日没有历史时直接采用典型日；光伏采用相同结构但 $a_{dk} = 1$ 。该收缩使冷启动平滑过渡到滚动历史估计，不需要把未来全年样本先用于训练。

图3展示 365 日净负荷 $P_{\text{load}} - P_{\text{pv}}$ 的季节和日内结构。正负区间交替说明光伏富余与负荷缺口都具有显著时变性；仅用单个年度均值无法形成可执行合同。

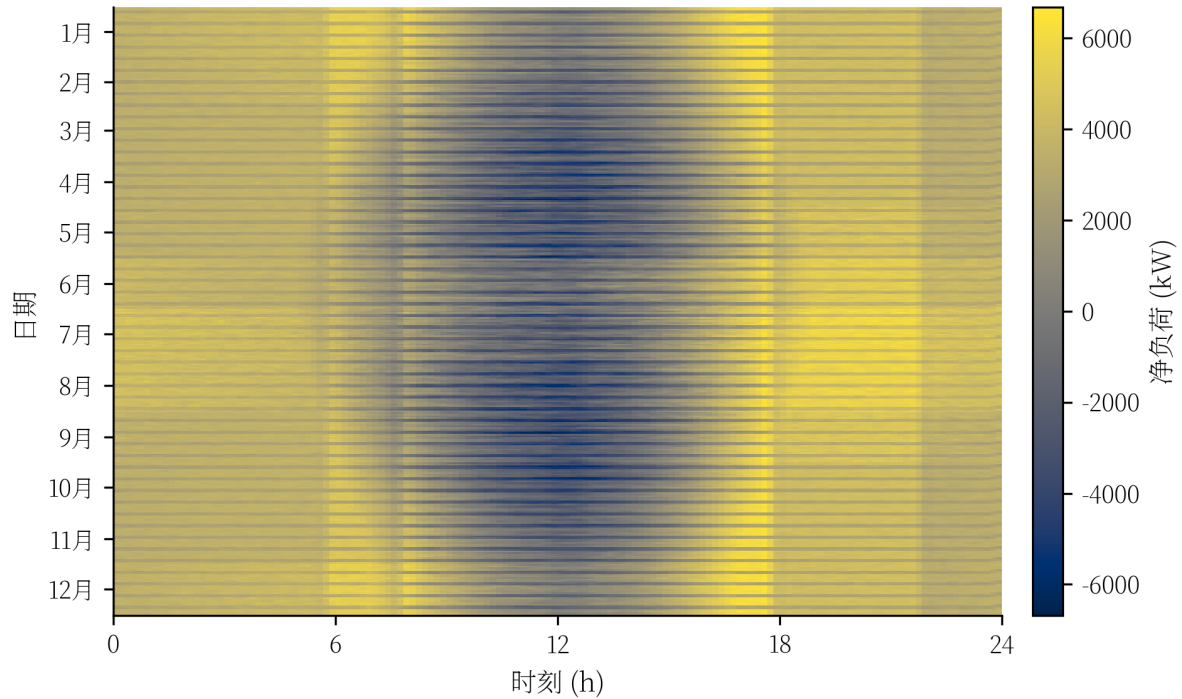


图 3 2025 年 365 日净负荷的季节-日内分布。该图使用全年原始数据，策略费用统计期另为 2 月至 12 月。

在正式统计期上，相似日负荷预测 MAE 为 722.5708 kW，RMSE 为 904.0899 kW；

光伏预测 MAE 为 335.9217 kW。图4表明多数点接近 45 度线，但仍存在明显尾部误差。因此，单纯把点预测当作确定真值会低估应急购电风险。

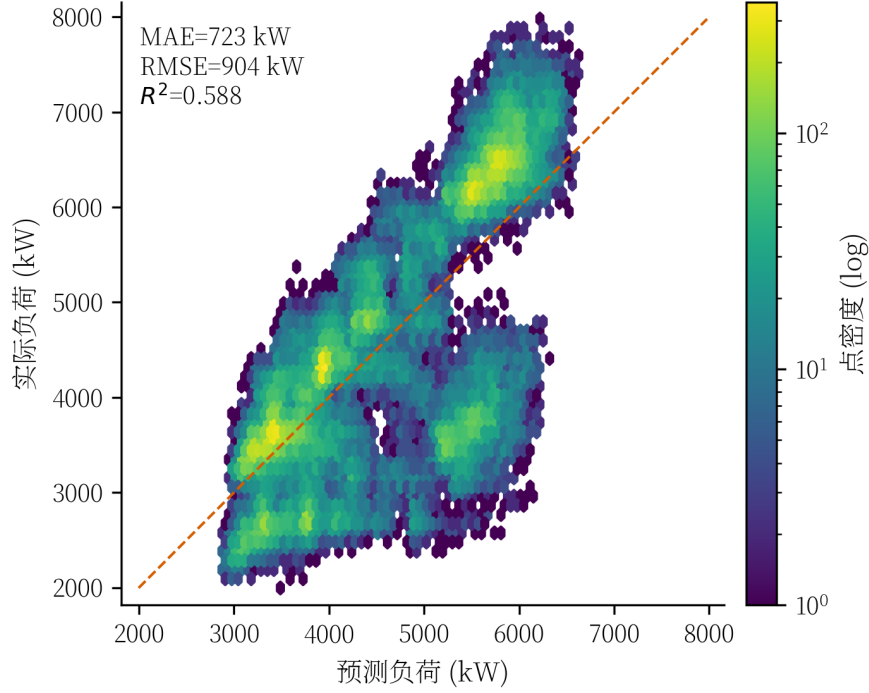


图 4 2 月至 12 月因果相似日负荷预测与真实负荷的比较。颜色表示样本密度，离 45 度线较远的点对应尾部误差。

4.2 联合残差特征与边际分位安全轨迹

定义负荷和光伏残差

$$e_{d,t}^L = L_{d,t} - \hat{L}_{d,t}, \quad e_{d,t}^V = V_{d,t} - \hat{V}_{d,t}. \quad (18)$$

在日期 d 只取最近 60 个已完成日期的残差，将一天的两类残差拼接为 288 维联合向量。正式风险模式不做聚类：每个历史残差日都作为等权经验样本，并把负荷、光伏残差同时叠加到当日点预测上。这样，净负荷偏高与可用光伏偏低的风险方向来自同一批联合残差路径，既保留两类误差的同期对应关系，也避免用未来日期估计当天风险。空历史退化为一个零残差样本，单条历史直接保留。

设第 s 个原始残差样本形成的负荷和光伏轨迹为 $L_t^{(s)}, V_t^{(s)}$ ，等权重为 $\pi_s = 1/|\mathcal{H}_d^{\text{res}}|$ 。对净负荷 $N_t^{(s)} = L_t^{(s)} - V_t^{(s)}$ 取加权经验 α 分位数，对光伏取 $1 - \alpha$ 分位数：

$$\tilde{N}_t = Q_\alpha(\{N_t^{(s)}, \pi_s\}), \quad (19)$$

$$\tilde{V}_t = \max\{Q_{1-\alpha}(\{V_t^{(s)}, \pi_s\}), -\tilde{N}_t, 0\}, \quad (20)$$

$$\tilde{L}_t = \tilde{V}_t + \tilde{N}_t. \quad (21)$$

经验分位数采用累计权重首次达到阈值的样本，不作线性插值。主方案在查看年度结果之前冻结 $\alpha = 0.8$ ；风险模式比较后也不反向调整该参数。式(21)分别沿净负荷上侧与光

伏下侧提取风险特征，再重构得到 $\tilde{L}_t, \tilde{V}_t \geq 0$ 且净差一致的单条轨迹。本文将其固定称为“联合残差特征提取后的边际分位安全轨迹”。它不是联合机会约束：联合路径只用于特征提取，进入 LP 后仍是一条逐槽边际分位轨迹，因而不提供跨时段联合概率保证，也不应称为完整随机 MPC。

从实现顺序看，每个日期包含四个确定步骤：先由已完成日期生成相似日点预测；再把最近 60 个已完成日期的原始联合残差逐条叠加到点预测；随后按式(21)逐槽取分位并重构安全轨迹；最后以当天真实 SOC 为初态求解 144 槽 LP。当天结束后才把新残差加入下一日历史。该“先预测、后观察、再更新历史”的顺序保证没有因为一次性载入全年附件而读取未来行。

作为压缩消融，另将 288 维联合残差按各维历史标准差标准化，并用固定随机种子 2026、10 次初始化的 K -means 压缩为至多 9 个加权代表样本；标准差小于 10^{-9} 时置为 1。聚类或未聚类两支均固定 $\alpha = 0.8$ ，使用相同的预测、执行与评价期，因此比较只检验“是否压缩原始联合残差”。正式方案仅在问题二和问题三上同时满足覆盖率不降低、Pinball 损失不增加、现金费用不增加、应急购电不增加四项事前裁决条件时才选择未聚类，否则保留聚类。结果显示未聚类在两支上均通过四项条件，故它成为正式主线；聚类只保留为消融证据。

4.3 日前合同优化与真实执行

在日期 d 的 0 时，用安全轨迹 $(\tilde{L}_t, \tilde{V}_t)$ 代替未知真实轨迹，求解统一物理模型。固定电价下，规划目标写为

$$\min \sum_{t=0}^{143} c_t g_t^P - \lambda_d S_{144}^P, \quad \lambda_d = \eta_d \text{median}\{c_t : t \text{ 属于剩余规划域}\}. \quad (22)$$

末端价值 $\lambda_d S_{144}^P$ 防止算法在一天末尾无代价地放空电池，但报告现金费用时不扣除这项信用。优化完成后合同 g_t 全天不再改变。

真实槽到达时，执行层只使用当前真实负荷和光伏。若光伏及合同供给有余量，则先在功率和 SOC 允许范围内充电，之后的合同剩余计为闲置合同，光伏剩余计为弃光；若有缺口，则先放电，仍不能满足的部分以 5 倍当前电价应急购电。这个优先级是透明的因果反馈规则，而不是用全天真实量重新优化。

规划层与执行层分离有一个重要含义：LP 给出的充放电计划服务于安全轨迹，真实执行动作则必须服从已实现的供需和当前 SOC。若真实光伏高于保守轨迹，额外余电优先充电；若真实负荷高于轨迹，电池可能比计划更早放电。因而“计划合同总量”与“实际购电供能量”不完全相同，现金费用也不能由计划目标值直接替代，必须逐槽按真实执行和应急量重新结算。

4.4 年度结果与基线比较

从 2 月 1 日至 12 月 31 日，无日内调账主策略现金费用为 18446381.6856 元，其中普通合同结算为 15383249.0906 元，应急费用为 3063132.5950 元；应急购电量为 499461.6179 kWh，闲置合同电量为 4217462.4412 kWh，弃光为 1977439.0052 kWh。年

末 SOC 为 10645.7848 kWh。每日费用和应急量具有明显异质性，见图5，图中的 7 日中心均线仅用于事后描述，不进入任何日前预测。

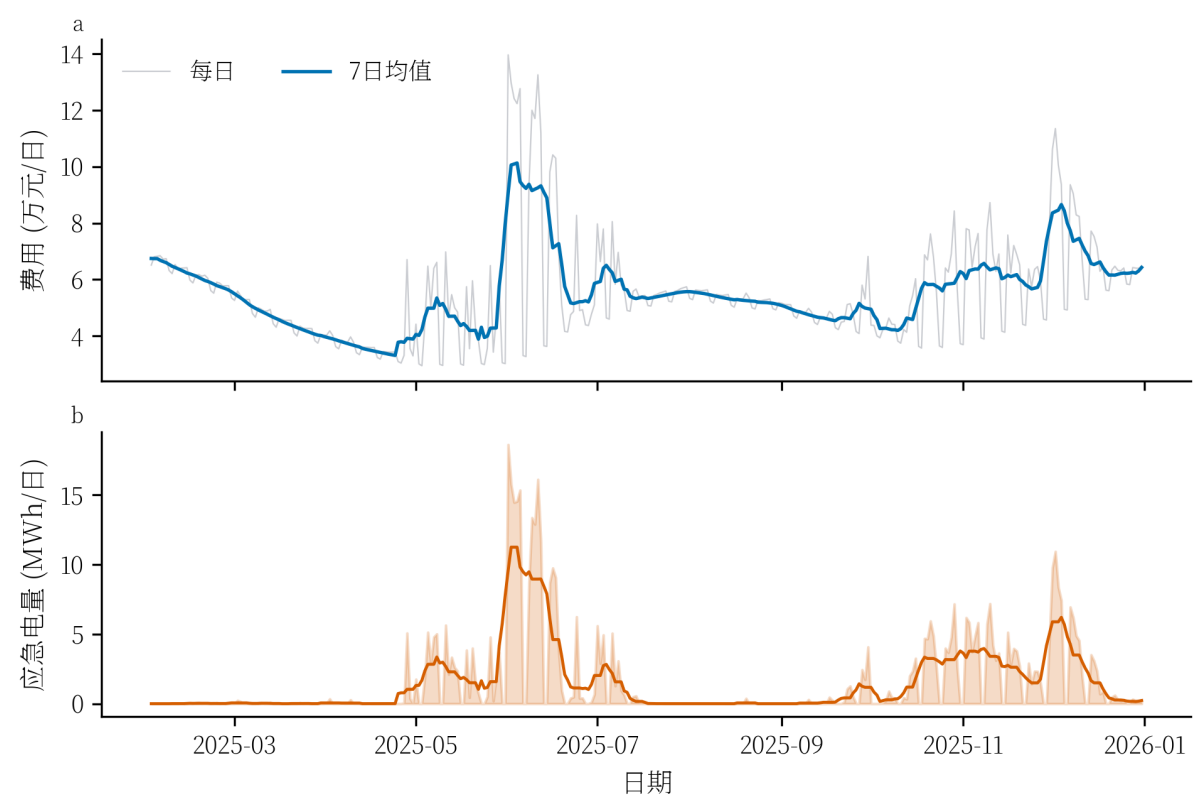


图 5 问题二 334 日现金费用与应急购电量。原始日值显示尾部风险，中心均线不作为模型输入。

为判断保守轨迹的作用，比较点预测、典型日和无储能三类基线。表4显示，点预测和典型日方案都因应急购电显著增加而费用较高。无储能方案费用低于这两项，却仍高于保守轨迹主策略；它的应急量与主策略相近，但弃光更高，说明储能和合同保守度必须联合考察，不能只看应急电量。

表 4 问题二主策略与基线的 334 日结果

策略	现金费用/元	应急购电/kWh	弃光/kWh
联合残差分位轨迹	18,446,381.69	499,461.62	1,977,439.01
点预测	27,574,292.97	2,685,630.97	1,618,696.22
典型日	27,753,076.30	2,724,419.83	1,773,185.71
无储能	21,648,775.09	540,700.12	3,058,159.92

图6进一步把合同结算与应急费用分开。主策略普通合同部分较高，是对预测尾部风险的事前支付；点预测和典型日虽然减少合同购买，却把更多缺口暴露在 5 倍应急价格下。无储能方案没有充放损耗，但中午光伏难以跨时段转移，其弃光约 305.82 万 kWh，显著高于有储能主策略。由此，主策略的优势不是单纯“买得更多”或“电池越大越好”，而是合同保守度与储能转移共同改变了高价缺口的位置和规模。

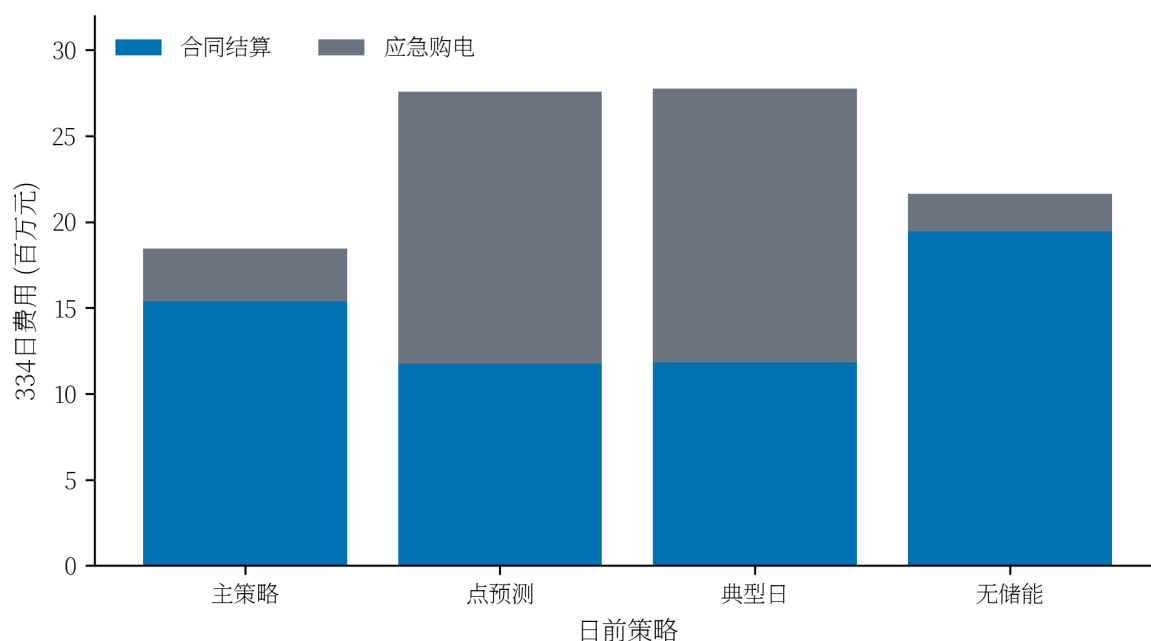


图6 问题二不同基线的合同费用与应急费用分解。柱高为334日总额，不表示抽样均值的置信区间。

图7给出正式风险模式裁决。问题二中，未聚类相对聚类把覆盖率由0.68309提高到0.70436、Pinball损失由232.289降至224.509 kW、费用由1876.42万元降至1844.64万元、应急购电由57.81万降至49.95万 kWh；问题三的四项指标也同向不劣。由于两支均通过预先规定的四项条件，正式结果采用未聚类模式；这里没有根据结果反调 $\alpha = 0.8$ 。

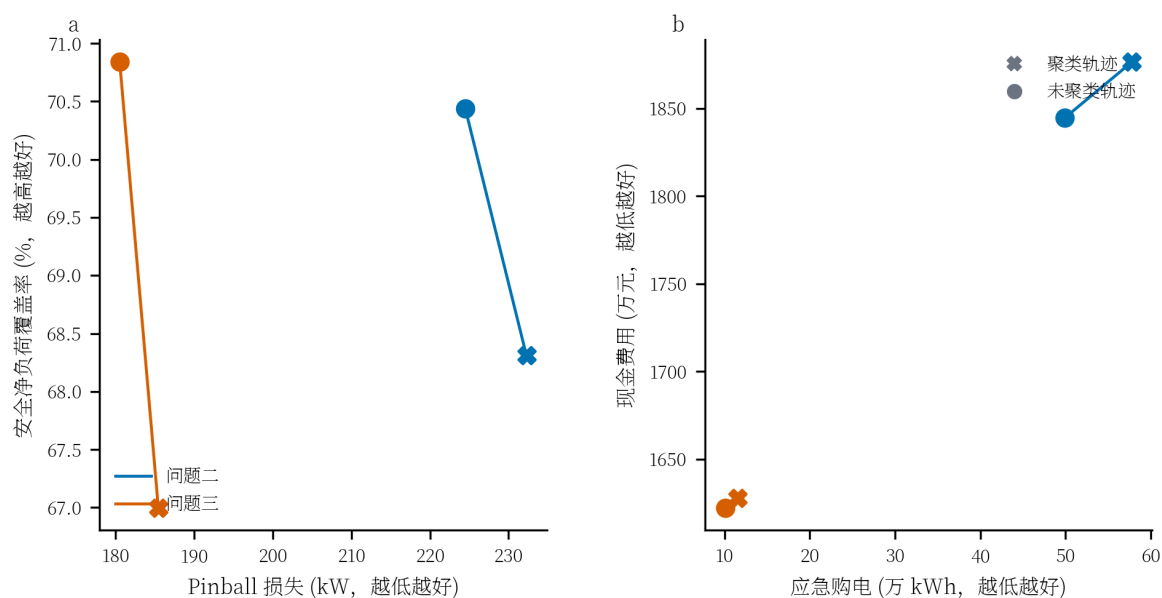


图7 问题二、三在固定 $\alpha = 0.8$ 下的风险模式消融。未聚类原始残差在两支上均通过覆盖率、Pinball 损失、费用和应急量四项裁决。

四个代表日的指定购电、储能和应急表分别列于附录A。代表日结果同时显示一个重要边界：策略在总体上降低风险，并不意味着每个日期都严格优于其他策略；这种日

期异质性将在问题三中继续讨论。

5 问题三：日内光伏预报与合同调整

5.1 官方预报的因果处理

附件给出 0、6、12、18 时发布的逐小时光伏预报。对发布时刻 $h \in \{0, 6, 12, 18\}$ ，先将未来整点预报裁剪为非负，再以当前时刻已经观测的光伏端点为锚，线性插值到 10 分钟网格。1 月 1 日 0 时没有前一已观测端点，锚值取 0。这样既避免光伏出现负值，也防止插值曲线在更新时刻与真实已观测前缀脱节。

令 $F_{d,h,\ell}$ 为日 d 在时刻 h 对提前 ℓ 小时目标的官方预报，真实值为 $V_{d,h+\ell}$ 。偏差样本定义为

$$r_{d,h,\ell} = V_{d,h+\ell} - F_{d,h,\ell}. \quad (23)$$

在日期 d 进行修正时，只取最近 60 个已完成日期的同一发布时刻残差均值，不能使用当天尚未完成的残差。经非负载剪的修正预报为

$$\hat{V}_{d,h,\ell}^{\text{off}} = \max \left\{ 0, F_{d,h,\ell} + \frac{1}{|\mathcal{R}_{d,h}|} \sum_{k \in \mathcal{R}_{d,h}} r_{k,h,\ell} \right\}. \quad (24)$$

若历史为空，则修正量为 0。

图8给出原始预报误差随发布时间和提前期的变化。对相同的当日剩余目标，0、6、12、18 时原始 MAE 分别为 186.7347、129.0903、35.1985 和 0.00036 kW，修正后分别为 188.1883、130.9920、35.5854 和 0.00051 kW。修正降低了平均偏差的绝对值，但 MAE 并未改善；18 时剩余目标主要处于夜间，满足真实光伏大于 100 kW 的 MAPE 有效样本数为 0，故不报告 MAPE 数值。该结果说明“消除均值偏差”不等同于“降低绝对误差”。

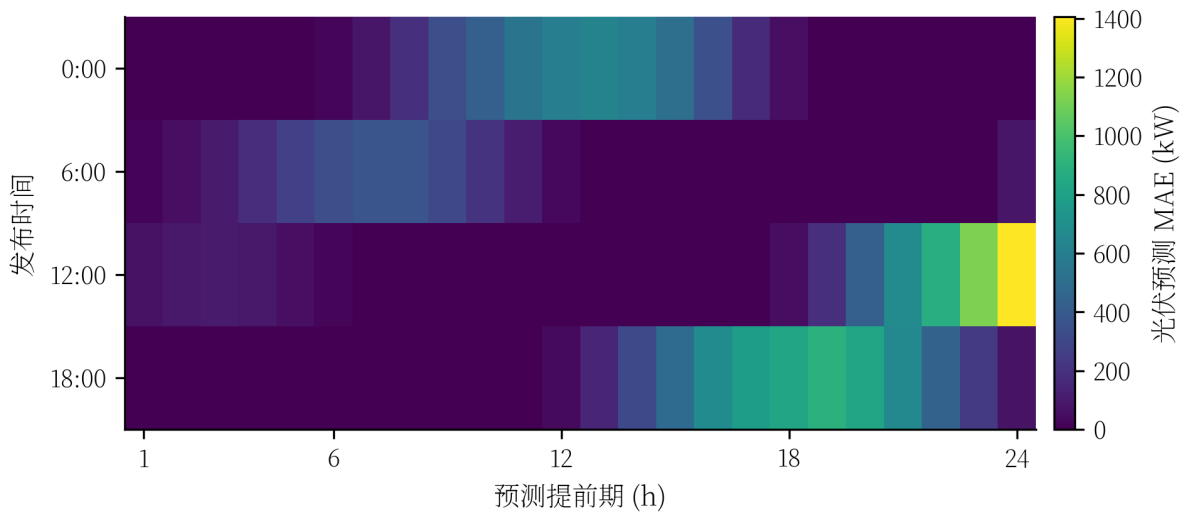


图 8 官方光伏预报 MAE 随发布时刻与提前期的变化。跨年不可得目标被排除，颜色不代表概率置信度。

5.2 滚动合同与结算模型

0 时先生成全天合同 p_t 。每次更新到达后，对 $t \geq 6h$ 、 $12h$ 或 $18h$ 的尚未执行时槽重新求解，已执行合同和实际能量流保持不变。规划中的负荷预测仍来自相似日模型；安全光伏轨迹用更新后的官方预报替代点预测，并继续使用过去残差形成保守余量。当天 SOC 从前一事件末端传递到下一事件，不因重规划而重置。

设 q_t 为某时槽最后一次更新后真正执行的合同。主结算解释以 0 时合同 p_t 为不可变基准，并按最终合同只结算一次：

$$C_t = c_t [\min(p_t, q_t) + 1.5 (q_t - p_t)^+ + 0.5 (p_t - q_t)^+] + 5c_t u_t. \quad (25)$$

其中 p_t 为 0 时已经支付的基准合同，最终合同中保留量按原价结算，下调量按 0.5 倍价格计调整费用，上调量按 1.5 倍价格计费；多次中间修改不逐次重复收费。为检验题意解释的影响，再对同一物理执行轨迹计算

$$C_t^{\text{alt}} = c_t [p_t + 1.5 (q_t - p_t)^+ + 0.5 (p_t - q_t)^+] + 5c_t u_t. \quad (26)$$

式(26)把下调量作为额外费用而非退款。它只用于同策略重新计费，没有在该规则下重新优化；在允许合同闲置且无退款收益的情况下，下调动作甚至可能被不下调严格支配。因此，两种费用必须分别解释，不能混称为同一个最优问题。

以 $p_t = 10 \text{ kWh}$ 、 $c_t = 1 \text{ 元/kWh}$ 且无应急购电为例，最终合同 $q_t = 5, 10, 15 \text{ kWh}$ 时，主结算费用分别为 7.5、10 和 17.5 元；其中 $q_t = 5$ 的 7.5 元是 5 元保留合同加 2.5 元下调调整费用，不是退款。替代结算分别为 12.5、10 和 17.5 元。该例说明两式只在下调区间不同：主式将下调量按半价计入调整后的合同费用，替代式则在原合同费用上再加半价下调费用。若不先明确这一差别，同一合同轨迹会得到不同的频率建议。

滚动求解还必须保持事件间状态一致。设相邻更新时刻为 $h_j < h_{j+1}$ ，在 h_j 只求解尚未执行的剩余段；区间 $[h_j, h_{j+1})$ 按真实值执行后，得到的 SOC 是下一次 LP 的固定初态。已执行合同、充放电和应急量均不回写。因而最终日费用可以按 144 个槽各自的最后有效合同相加，而不能把四次规划目标相加。

图9展示 6 月 21 日合同随预报更新发生有方向的变化。图中的实际净负荷尚未扣除储能动作，因此不能把合同曲线与净负荷曲线的差直接解释为应急量或弃光量。

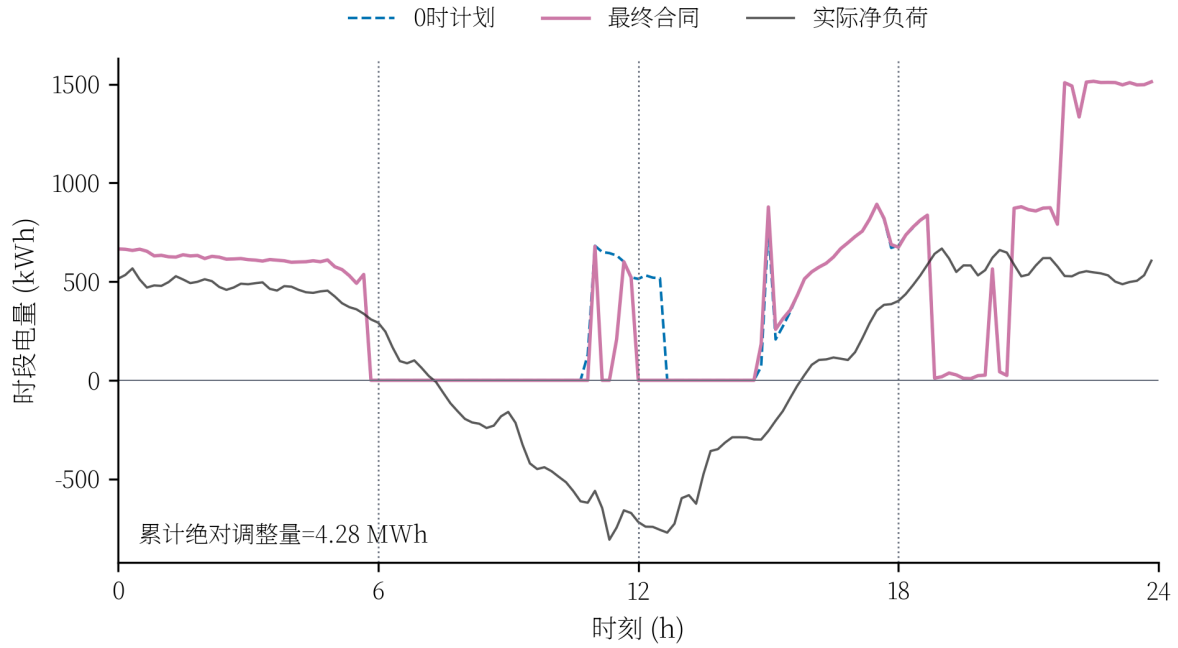


图 9 6 月 21 日 0 时初始合同与日内最终合同。竖线为 6、12、18 时更新事件。

5.3 主策略结果

问题三 334 日现金费用为 16222449.2957 元，应急购电量为 100703.5160 kWh，闲置合同为 3232703.5560 kWh，弃光为 1863872.6197 kWh，年末 SOC 为 10645.7847 kWh。相对问题二，现金费用减少 2223932.3899 元，应急购电减少 398758.1019 kWh。但这项差异同时改变了光伏信息来源、合同调整权和执行路径，并非“只增加更新次数”的纯频率效应。

四个代表日中，3 月 20 日、6 月 21 日和 9 月 23 日费用下降，而 12 月 21 日问题二费用为 63398.4227 元、应急购电为 472.7868 kWh，问题三费用为 66531.3792 元、应急购电为 1077.7471 kWh。因此，年度改善不能被写成逐日占优。详细代表日购电、储能与应急结果见附录 A。

5.4 相同 0 时信息下的更新频率分析

为尽量分离“0 时信息不同”的影响，固定 0 时采用问题三的同官方信息，比较只在 0 时、0/12 时、0/6/12/18 时和每 2 小时更新四组方案。2 小时情形并非附件提供的新官方预报：在非官方发布时刻，模型沿用最近一次已发布预报，并以当天已经实现的最多 18 个 10 分钟槽偏差构造 3 小时指数衰减的合成短临更新；该更新只调整安全光伏，同时保持安全负荷轨迹，不重新标定场景概率。

四组费用及两种结算的对照列于表 5。负的“相对 6 小时节约”表示候选方案更贵，而不是负费用。

表 5 固定电价下相同 0 时信息的更新集合比较

更新集合	主结算费用/元	相对 6 小时节约/元	替代结算相对 6 小时节约/元
仅 0 时	17,346,919.25	-1,124,469.95	-321,227.29
0、12 时	16,837,256.36	-614,807.06	-409,913.73
每 6 小时	16,222,449.30	0	0
每 2 小时合成更新	15,594,064.56	628,384.73	9,074.01

2 小时方案在主结算下相对 6 小时方案节约 3.8738%，全年增加 2672 次更新。若每次额外更新的综合通信、计算和运维成本记为 k ，则其净收益为

$$B_{2h} = 628384.7338 - 2672k. \quad (27)$$

故盈亏平衡阈值为 $k^* = 235.1739$ 元/次。四个代表日并非均不劣，年度均值也不能替代逐日风险判断。替代结算下 2 小时相对 6 小时只节省 9074.0127 元，不能把主结算阈值移用于该口径。图10直观显示结算解释对更新价值的影响，因此合理建议是“在主结算成立且每次增量成本低于阈值时采用 2 小时合成更新”，而不是无条件追求最高更新频率。

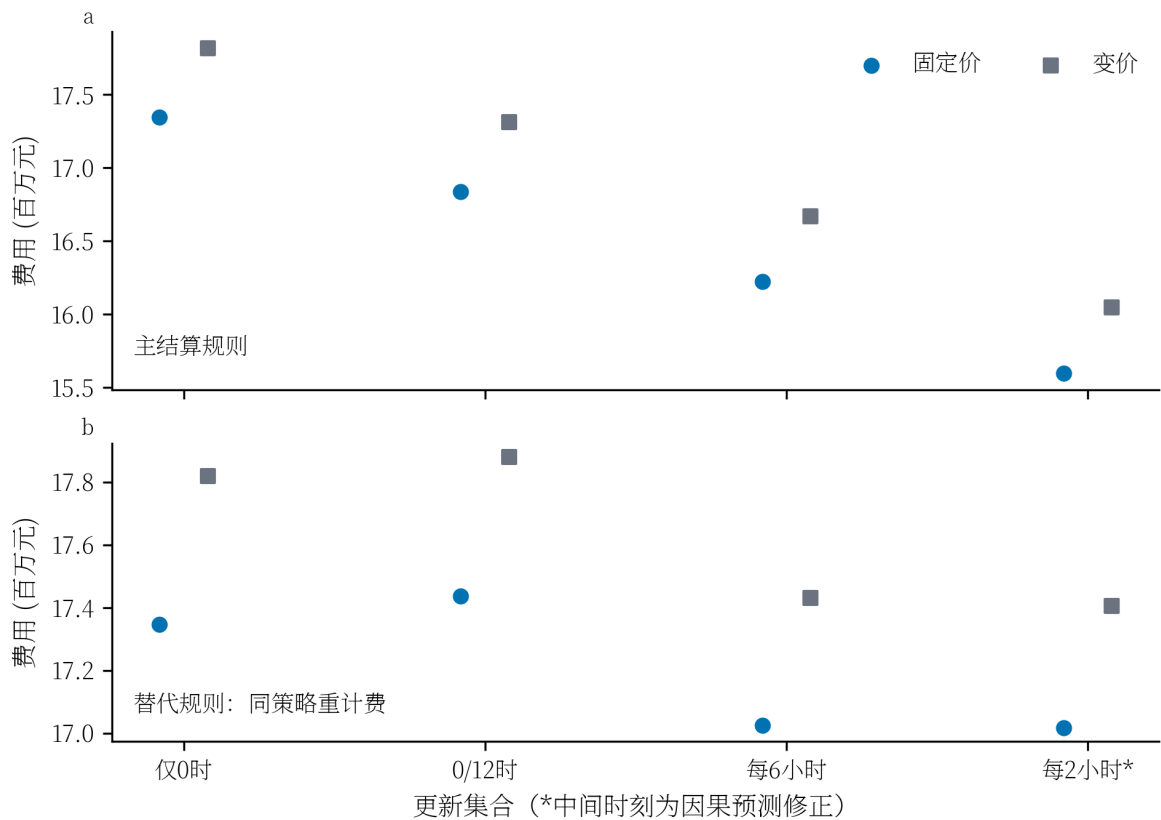


图 10 固定与变动电价下不同更新集合的总费用，分别按主结算和替代结算重计。2 小时方案使用因果合成更新。

为隔离官方预报版本可读性本身的价值，进一步固定 0 时信息、固定正式执行和重优化规则，只改变额外可读发布时间集合 $\mathcal{S} \subseteq \{6, 12, 18\}$ 。8 组方案都保留 0、6、12、

18 时四次重优化；当某时刻不属于 $\mathcal{S}$ 时，控制器仍按既定事件重求后续合同，但该时刻发布的官方预报及其历史校准残差均不可读，只能沿用当时信息集内的轨迹。全集相对冻结 0 时基线节省 381887.2843 元（2.299925%），应急购电减少 28433.2128 kWh；按三时刻 Shapley 分摊，6 时、12 时、18 时的贡献分别为 126227.8748、188849.9250 和 66809.4845 元。图11隔离的是信息可读性，不是问题二到问题三的总收益。

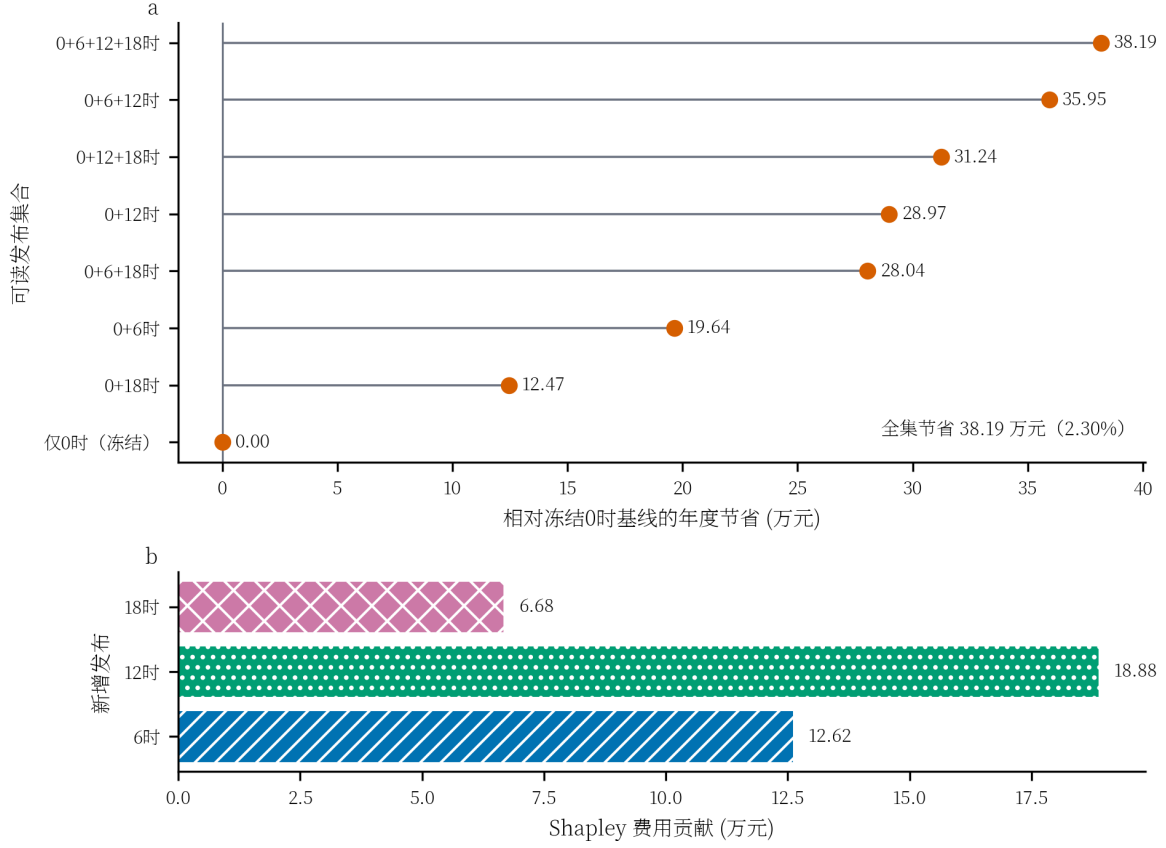


图 11 固定 0 时信息下官方光伏预报发布时间的 8 组信息消融与 Shapley 贡献。所有组均保留相同重优化时刻结构，屏蔽版本及其历史残差不可读。

6 问题四：变动电价下的滚动调度

6.1 未来价格的因果预测

问题四的实际电价随日期和时槽变化。若在规划时直接读取当天未来真实价格，会产生明显的未来信息泄漏。本文对日期 d 、槽 t 的 0 时价格预测取过去 30 日同槽价格的中位数：

$$\hat{c}_{d,0,t} = \text{median} \{c_{k,t} : \max(0, d - 30) \leq k < d\}. \quad (28)$$

首日没有历史时使用附件典型价格。日内时刻 h 重新规划时，计算当天已经实现的最近至多 18 槽价格偏差中位数 $b_{d,h}$ ，并按 3 小时尺度向未来衰减：

$$\hat{c}_{d,h,t} = \max \left\{ 10^{-4}, \hat{c}_{d,0,t} + b_{d,h} \exp \left[-\frac{t - t_h}{18} \right] \right\}, \quad t \geq t_h. \quad (29)$$

其中 $t_h = 6h$ 。规划目标使用 $\hat{c}$ ，真实 c 仅用于合同和应急费用结算。图12显示各月价格分布具有离散性和尾部，故单一月均价不足以描述调度机会；箱线图外的离群点均被保留。

中位数而非均值用于 0 时价格预测，是因为月内价格分布包含尾部，少数尖峰会明显拉动均值。该选择并不意味着忽略尖峰：安全供能仍由负荷和光伏分位轨迹控制，而价格中位数只决定有限电池能量在剩余时槽间的相对配置。当日偏差项使整体价格水平随已实现数据上移或下移，指数衰减则避免把刚刚发生的局部异常永久外推到全天。

在实际执行时，储能动作与真实价格的相关性只能作为事后描述，不能反向证明控制器预知价格。规划时可利用的是 $\hat{c}_{d,h,t}$ ；真实 $c_{d,t}$ 在槽结束时进入结算。若实际高价尖峰未被历史中位数和已实现偏差预见，策略可能保留不足的 SOC，这正是个别代表日有调账方案费用反而上升的可能机制之一。

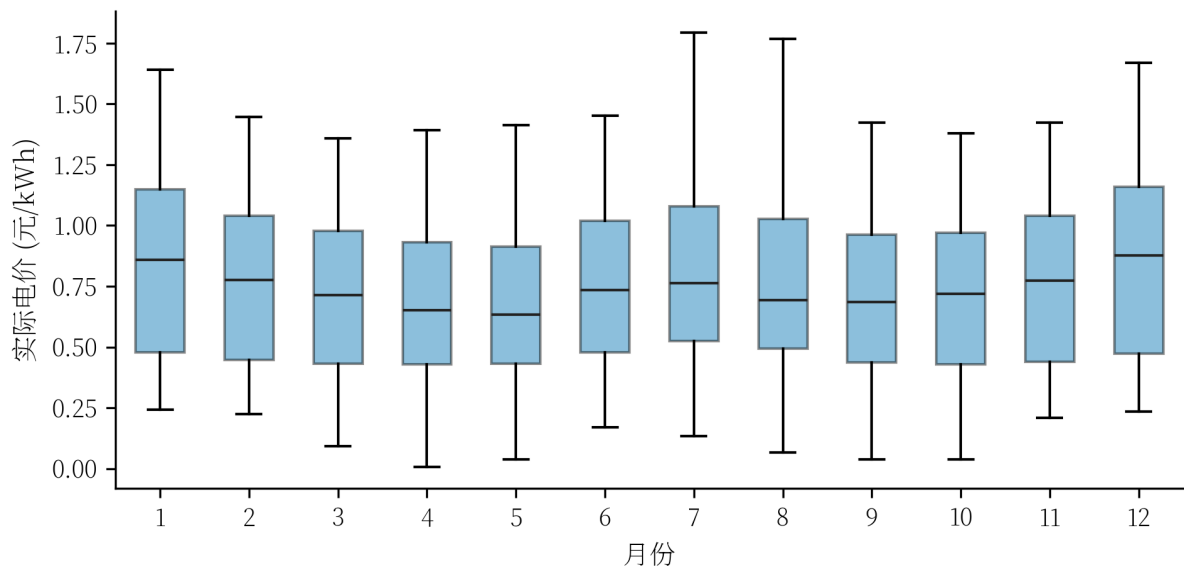


图 12 2025 年各月 10 分钟电价分布。箱体为四分位区间，须线为 1.5 倍四分位距，离群点未删除。

6.2 两种策略及结果

问题四的无调账方案沿用问题二的相似日与分位轨迹，只把固定价目标替换为式(28)的预测价格；有调账方案沿用问题三的 0、6、12、18 时更新机制，并在每次滚动求解时使用式(29)。两者执行层均只读取当前真实量，储能跨日连续。

表6给出正式统计结果。有调账方案比无调账方案节约 2253878.1999 元，应急购电减少 397918.6412 kWh。若把问题二和问题三也纳入同一图，固定价与变价都呈现“有调账总体费用更低”的结果，见图13。但是，固定价的 $q_2 \rightarrow q_3$ 和变价的 $q_{4_2} \rightarrow q_{4_3}$ 都同时改变了信息、调账权与执行路径，不能当作单因素频率效应。

表 6 问题四两个方案的 334 日结果

方案	现金费用/元	应急/kWh	闲置合同/kWh	弃光/kWh
无调账	18,924,373.20	498,851.05	4,198,994.13	1,992,637.63
有调账	16,670,495.00	100,932.41	3,226,185.39	1,869,483.00

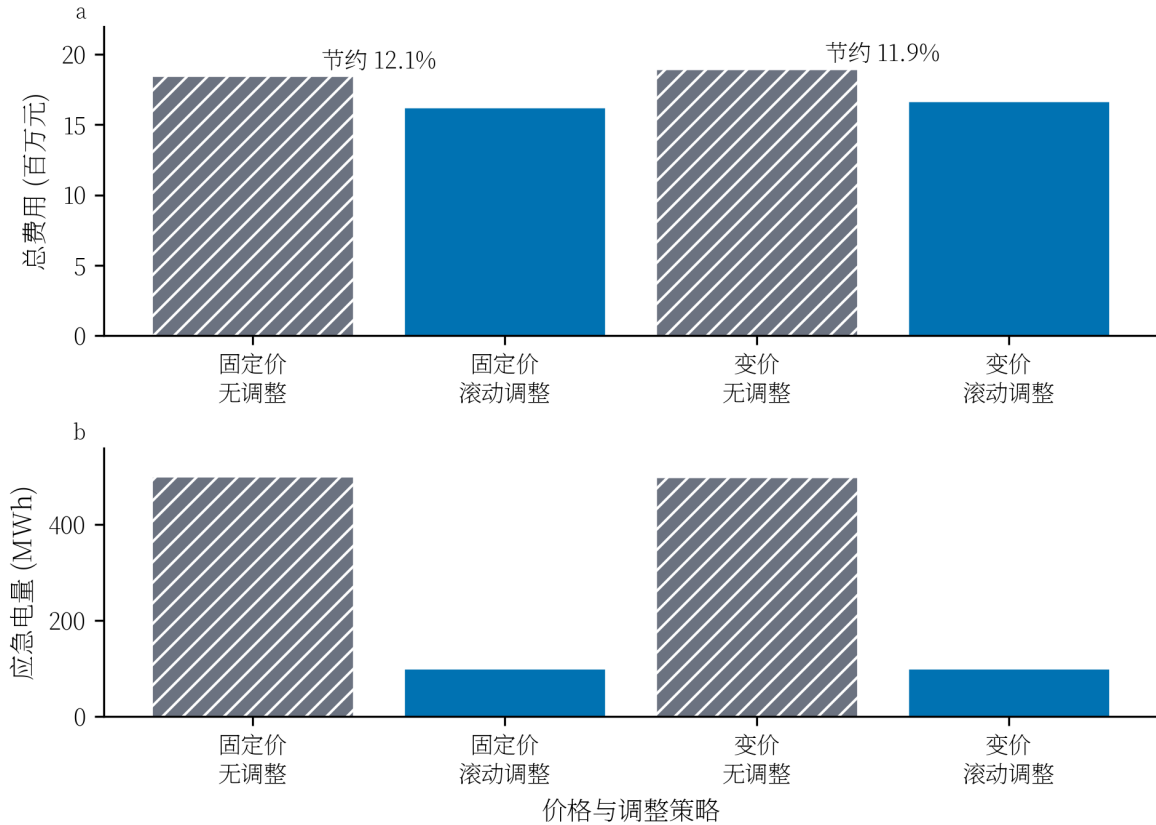


图 13 固定价与变动电价下四个主策略的 334 日费用和应急购电量。

代表日同样并非全部改善：12 月 21 日无调账方案费用为 74326.6564 元，有调账方案反而为 78804.6801 元；应急电量由 472.7868 变为 1057.3935 kWh。这说明滚动预测在总体上更有效，却可能在个别日期因预报误差、储能边界和价格预测偏差叠加而失利。两方案的完整代表日指定表见附录A和配套工作簿。

6.3 变价条件下的更新频率建议

在相同 0 时信息下，仅 0 时、0/12 时、每 6 小时和每 2 小时方案费用分别为 17819644.9894、17313769.2898、16670494.9986 和 16044509.7501 元。2 小时方案相对 6 小时方案节约 625985.2485 元，即 3.75505%。两方案相差 2672 次更新，故净收益为

$$B_{2h}^{\text{var}} = 625985.2485 - 2672k, \quad (30)$$

盈亏平衡阈值为 234.2759 元/次。若采用替代结算式(26)对同一策略重计费，节约额缩小为 25502.8132 元，不能把主结算阈值直接移用于该口径。

因此，问题四的建议为：在题设主结算解释成立、能够可靠取得最新官方预报和已观测前缀，且单次增量更新成本低于约 234.28 元时，可采用 2 小时合成更新；否则 6 小时官方发布节奏具有更稳健的实施性。2 小时结果仅表示本题数据上的因果短临更新价值，不代表系统获得了额外官方预报产品。

7 模型检验、敏感性与稳健性分析

7.1 物理可行性与因果性检验

对问题一及四个年度主策略逐槽检查规划层式(2)、式(3)与执行层式(7)、式(8)，并核对 SOC 边界、充放电功率、非负性、同时充放和跨日连续性。正式主策略的最大真实母线平衡残差为 4.55×10^{-13} kWh；5010 个规划 LP 全部返回 **optimal**，最大 LP 等式残差为 2.57×10^{-11} kWh。规划层与执行层的最大同时充放量均为 0。这些量接近浮点运算精度，说明输出满足本文两层物理方程。

因果性检验在第 0、1 和 364 日的 0、2、...、22 时，对未来负荷、光伏、价格和未发布预报分别作 36 组扰动。当前预测、价格估计和当前 LP 计划的最大差均为 0；已执行前缀和 SOC 边界也不受未来扰动影响。这验证了实现遵守信息集 $\mathcal{F}_{d,h}$ 。该检验说明“没有使用被扰动的未来数据”，但不是对所有可能代码路径的形式化证明。

7.2 54 组实验设计

实验共 54 组，其中 16 组为固定价或变价条件下的安全轨迹、点预测、典型日、无储能，以及 0/12/6/2 小时更新基线；另有 38 组单因素实验。单因素范围包括：

- 保守分位参数 $\alpha = 0.70, 0.75, 0.80, 0.85, 0.90, 0.95$ ；
- 聚类压缩消融的簇数上限 5、9、12、16，以及残差历史 30、60、90 日；
- 储能容量和功率分别为主值的 0.8、1.0、1.2 倍；
- 往返效率从 0.81 提高到 0.90，即单向效率改为 $\sqrt{0.9}$ ；
- 应急电价倍率 3、4、5、6、8。

正式主模型始终采用未聚类原始残差；簇数扫描只评价压缩对照，不能解释为主模型调参。容量改变时，SOC 下限、上限和初态仍分别为容量的 10%、90% 和 50%，而功率与效率保持不变；功率实验则不改变容量。应急倍率实验固定物理策略和 $\alpha = 0.8$ ，只重新计算费用暴露，因此不能称为各倍率下重新寻优的最优策略。

7.3 规划—执行一致性诊断

LP 中的连续变量仅包括合同、规划光伏利用、规划充放电、目标 SOC；允许调账时再加入合同调整差分。应急购电 u_t 和闲置合同 w_t 只在真实量到达后由因果贪心反馈确定，是执行与结算记账量，不是 LP 决策变量。由此，计划目标 SOC 与真实执行 SOC 出现偏差是预测误差经反馈层吸收的结果，并不表示违反状态方程。

四个正式主分支的目标 SOC 日均绝对偏差分别为：问题二 1272.24kWh、问题三 801.76 kWh、问题四无调账 1320.57 kWh、问题四有调账 840.82 kWh。有调账分支偏差较小，说明日内新信息使后续计划更贴近真实执行；但偏差不为 0 也表明滚动合同层不能替代执行反馈层。本文据此把整体框架固定称为“合同层滚动线性规划 + 因果贪心反馈”。

7.4 保守参数与设备参数

图14展示 α 对费用和应急量的权衡。固定价问题二的测试点中，最低费用出现在 $\alpha = 0.90$ ；变价有调账方案的测试点中，最低费用出现在 0.8。主参数 0.8 在查看年度结果之前已经冻结，本文不因样本内曲线反调，也不把任一测试点宣称为样本外普适最优。曲线只用于说明保守度改变会同时影响合同过购、应急缺口和弃光。

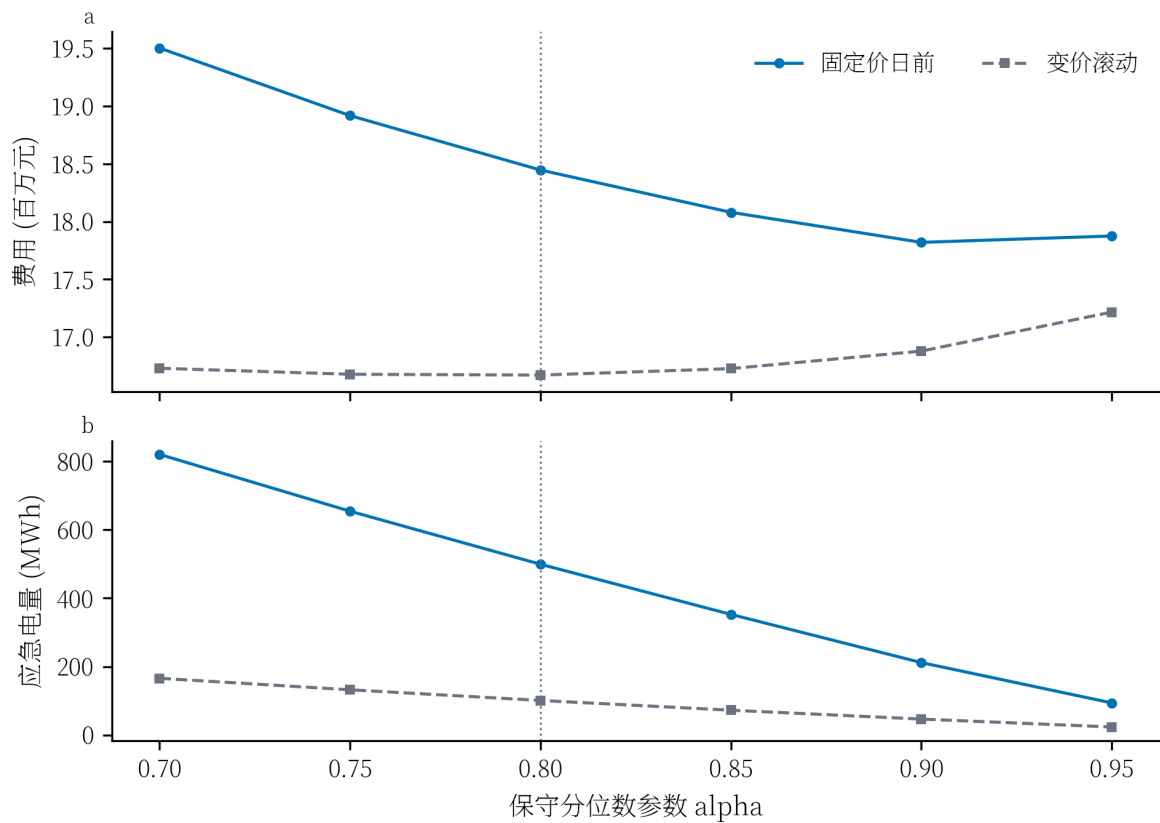


图 14 问题二与问题四有调账策略对保守分位参数的单因素敏感性。主方案 $\alpha = 0.8$ 为事前冻结值，曲线不用于事后反调。

表7列出若干有代表性的单因素结果。历史窗口 30 日时，问题二和问题四有调账方案费用分别为 1678.29 万元和 1628.20 万元，反映 2025 年样本中的近期结构可能更有用，但不证明短历史在其他年份总是更好。容量增加 20% 后，两方案费用分别为 1790.53 万元和 1594.44 万元；提高往返效率则将费用降至 1800.49 万元和 1613.88 万元。所有比较都只改变表中一个因素，其余参数保持主设定。

表 7 部分单因素实验的 334 日现金费用（万元）

实验设置	问题二无调账	问题四有调账
主设定	1844.64	1667.05
残差历史 30 日	1678.29	1628.20
储能容量增加 20%	1790.53	1594.44
储能功率降至 80%	—	1663.65
往返效率提高到 90%	1800.49	1613.88

问题四有调账方案把功率降至主值的 80% 时，样本内费用约为 1663.65 万元，略低于主方案 1667.05 万元。这一非单调性并不违反物理规律：控制器采用安全轨迹规划和真实值下的贪心执行，并非在完全信息下对全年重新全局优化；更大的设备可行域不会自动保证该滚动反馈策略产生更低的事后结算费用。所有容量实验都从容量的 50% 起步，SOC 上下界同步保持 10% 和 90%；1 月因果预热后连续传递状态，正式费用仍只统计 2 月至 12 月。

7.5 月内经验低光伏分层

为避免全年统一阈值把季节变化误判为天气效应，本文在每个月内部按当日实际光伏电量的 20% 经验分位数分层，把不高于阈值的日期称为“月内经验低光伏日”。每个主分支均有 72 个低光伏日和 262 个其他日期。图15报告两组的平均日费用与平均应急量。

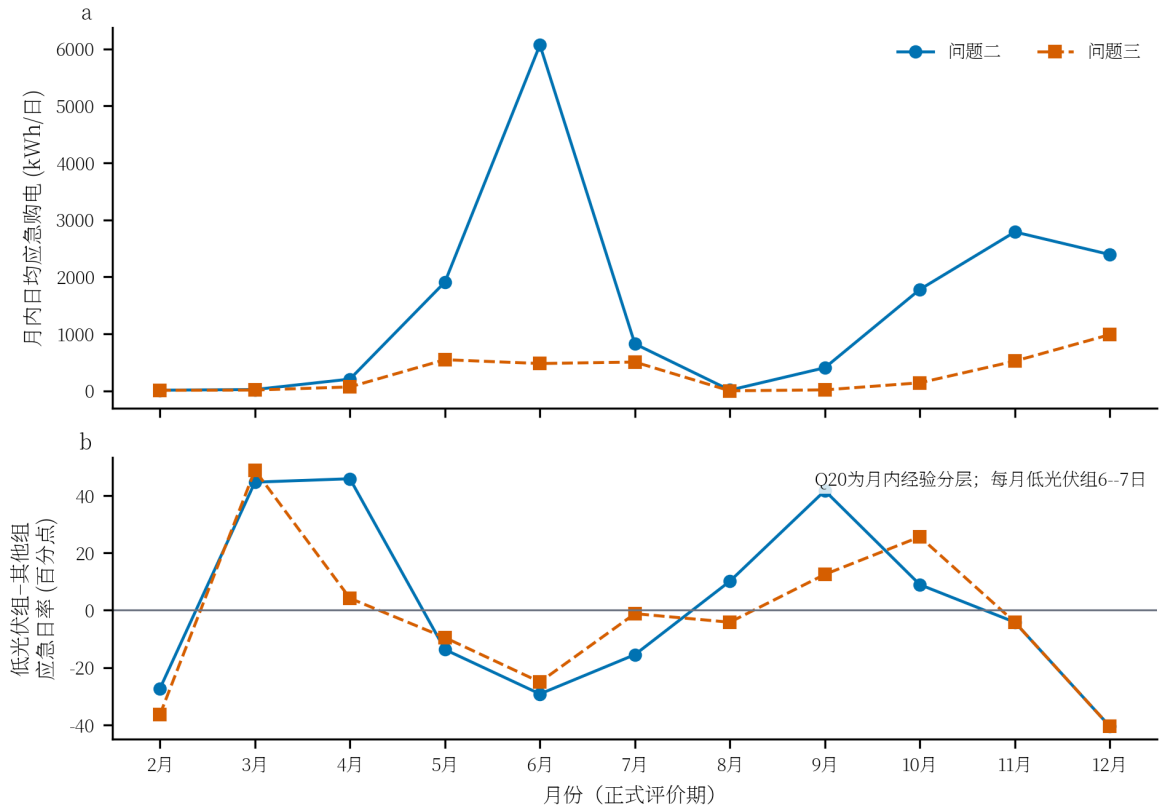


图 15 四个正式主分支在月内经验低光伏日与其他日期上的费用和应急量。分层按各月实际光伏日总量的 20% 经验分位数定义。

问题二低光伏组和其他组的平均日费用分别为 57685.58 元和 54553.51 元，平均应急量分别为 1738.11 和 1428.69 kWh；问题四无调账对应 59150.65 元、55975.29 元和 1718.36、1431.79 kWh。两类有调账策略的低光伏组平均日费用同样更高，但平均应急量反而略低：问题三为 50487.26 元、48043.38 元和 256.85、313.78 kWh，问题四有调账为 51831.62 元、49384.04 元和 254.06、315.42 kWh。该现象说明费用暴露同时受合同、价格、SOC 和应急共同影响。由于分层使用真实光伏且只做月内事后比较，本文不将其称为“阴天效应”，也不作跨月因果归因。

7.6 结算与时间口径敏感性

同一策略在主结算和替代结算下费用差异明显。问题三主策略由 1622.24 万元增至 1702.57 万元，问题四有调账策略由 1667.05 万元增至 1743.15 万元。更重要的是，固定价下 2 小时相对 6 小时的节省从 62.84 万元缩小为 0.91 万元，变价下从 62.60 万元缩小为 2.55 万元。因此，更新频率结论必须与式(25)绑定，不能只给一个脱离合同条款的百分比。

时间口径实验把典型日全部槽循环平移 10 分钟。购电总量仅有约 2.38×10^{-9} kWh 的浮点差，而费用下降约 0.10 元。结果表明年度策略量级不依赖这一微小映射，但题目指定时段数值和逐槽费用仍需要统一定义，本文采用区间终点解释。

7.7 计算规模与结果可追溯性

每天的规划域最多为 144 槽，随日内时间推进逐步缩短。每槽 LP 包含合同、规划光伏利用、规划充放电与目标 SOC；调账分支再包含相对 0 时合同的调整差分。约束数量随剩余槽数线性增长，所有目标和约束均为线性形式。未聚类正式模式直接从最近 60 个已完成日期提取边际经验分位数，不含随机步骤；聚类只作为 288 维联合残差的压缩消融，固定种子 2026 和 10 次初始化。

为了使结果可追溯，每个策略同时保留三层输出：第一层是逐槽规划合同、目标 SOC 与真实能量流，用于分别检查式(2)、式(3)和式(7)、式(8)；第二层是每日合同量、现金费用、应急量、闲置合同和弃光；第三层是 334 日总量及实验参数。论文汇总与五份题目工作簿均从同一逐槽数据源聚合。复现记录只登记文件路径、大小和修改时间，不计算或比较文件哈希，因此不提供密码学完整性保证。

结果解释遵循“数值可行、因果可用、外部有效性待检验”的层次。逐槽残差接近机器精度证明计算结果满足本文方程；未来扰动不影响当前决策说明实现遵守给定信息边界；54 组实验和分层诊断说明结论在主要设定变化下如何移动。但这些证据不能自动证明另一年份、另一微网或另一合同条款下仍有同样节约率。工程迁移时应先复现输入映射和结算，再用滚动样本外数据重新校准历史窗口与保守度。

7.8 模型优点与局限

本文方法具有四点优点：第一，信息集明确，预测与执行均可通过未来扰动检验；第二，规划层为连续 LP，执行层为可逐槽复核的因果规则；第三，联合残差同时保留负荷和光伏误差的同期特征，未聚类正式模式又避免压缩损失；第四，风险模式、信息版本、结算口径、更新集合和设备参数都有真实重算或明确标注的同策略重计费证据。

局限也十分明确。其一，边际分位安全轨迹会丢失跨时段联合概率，不能提供机会约束保证。其二，相似日和历史中位数属于低参数预测，可能无法捕捉节假日、极端天气或结构突变。其三，真实执行采用优先级规则而非全年完美信息优化，设备参数的实际费用可能非单调。其四，储能老化、备用、配网潮流和需求响应不在题设数据中，本文没有虚构这些成本；工程实施时应加入退化成本、网络约束和滚动终端集合。其五，2 小时方案是由已发布预报与观测构成的合成短临结果，不能替代对真实高频预报服务的独立评估。

8 结论与实施建议

本文用统一的 10 分钟规划与执行方程连接四个问题，并将“当时可得信息”作为滚动决策的硬边界。核心方法是“合同层滚动线性规划 + 因果贪心反馈”：规划层基于预测轨迹确定合同与目标 SOC，执行层在真实量逐槽到达后完成光伏消纳、充放电和应急记账。

第一，典型日连续 LP 得到购电 59482.6982 kWh、费用 35126.9489 元。在 6000 kWh 循环 SOC 约束下，储能完成低价充电与高价放电，给出了后续全年模型的物理和量纲基线。

第二，固定价格且不能日内调账时，最近 60 个已完成日期的原始联合残差被用于

提取“联合残差特征提取后的边际分位安全轨迹”。正式模式不聚类、 $\alpha = 0.8$ 事前冻结, 所得 334 日现金费用为 1844.64 万元, 应急购电 49.95 万 kWh。与固定参数的聚类压缩对照相比, 未聚类模式在问题二和问题三上均同时满足覆盖率不降低、Pinball 损失不增加、费用不增加和应急量不增加, 因而被选为正式主线; 这项模型仍是确定性分位轨迹 LP, 不是完整随机 MPC 或机会约束。

第三, 在 0、6、12、18 时接收官方光伏预报并调整未执行合同时, 问题三费用为 1622.24 万元, 应急购电为 10.07 万 kWh; 相对问题二减少 222.39 万元和 39.88 万 kWh, 但该总差异同时包含信息来源、调账权和执行路径变化。固定四个重优化时刻、只屏蔽额外官方版本的信息消融表明, 全集相对冻结 0 时节省 38.19 万元、应急减少 2.84 万 kWh; 6、12、18 时信息的 Shapley 贡献分别为 12.62、18.88 和 6.68 万元, 其中 12 时贡献最大。这才是额外官方预报版本在当前实验定义下的边际信息价值。

第四, 相同 0 时信息下, 固定价 2 小时合成更新比 6 小时更新节省 62.84 万元, 即 3.8738%; 变价情形节省 62.60 万元, 即 3.7551%。两者分别对应 235.17 元/次和 234.28 元/次的增量更新成本阈值。替代结算下, 节省分别缩小为 0.91 万元和 2.55 万元, 说明频率建议高度依赖合同下调条款, 不能把主结算的阈值直接外推。

第五, 变动电价下, 未来价格只由过去 30 日同槽中位数和当日已实现偏差估计, 真实价格只参与事后结算。无调账与有调账策略费用分别为 1892.44 万元和 1667.05 万元, 应急购电分别为 49.89 万和 10.09 万 kWh; 有调账总体节省 225.39 万元并减少 39.79 万 kWh 应急购电。代表日中仍存在费用和应急同时变差的日期, 说明年度优势不等于逐日占优。

数值与稳健性证据形成了完整闭环: 5010 个规划 LP 全部达到 **optimal**, 最大 LP 等式残差为 2.57×10^{-11} kWh, 最大真实母线平衡残差为 4.55×10^{-13} kWh, 规划与执行均未出现同时充放; 54 组实验、规划-执行 SOC 偏差和月内经验低光伏分层进一步揭示了模型的适用边界。这些证据支持题设数据内的可行性、因果性和机制解释, 但不构成跨年份概率保证。

实施上, 建议先把 6 小时官方发布节奏部署为影子调度, 逐日核对预测误差、结算条款、合同闲置和 SOC 轨迹。在主结算得到确认、数据接入稳定且单次增量更新成本低于约 234 元时, 再试行 2 小时合成短临更新; 数据缺失或估计收益不足时回退到最近一次官方发布方案。若获得多年份天气、节假日和真实高频预报, 应采用滚动样本外检验重新评估 α 、历史窗口和更新频率, 并补充储能退化与配电网约束。

AI 工具使用声明

本文参考稿在资料整理、代码检查、结果表述、LaTeX 排版和语言润色阶段使用了人工智能辅助工具。核心模型采用的变量、约束、因果信息边界、实验口径及最终是否采纳, 均需由参赛队伍结合题目和原始附件逐项判断。人工智能生成内容可能存在错误, 本参考稿不代表队伍已经完成人工复核, 也不能未经理解、核验和改写直接提交。所用工具、主要用途、交互示例、采纳范围与待人工核验事项另见支撑材料中的《AI 工具使用详情》。

参考文献

- [1] Jiefeng Hu, Yinghao Shan, Josep M. Guerrero, Adrian Ioinovici, Ka Wing Chan, and Jose Rodriguez. Model predictive control of microgrids – an overview. *Renewable and Sustainable Energy Reviews*, 136:110422, 2021.
- [2] Hao Liang and Weihua Zhuang. Stochastic modeling and optimization in a microgrid: A survey. *Energies*, 7(4):2027–2050, 2014.
- [3] Stefano Raimondi Cominesi, Marcello Farina, Luca Giulioni, Bruno Picasso, and Riccardo Scattolini. A two-layer stochastic model predictive control scheme for microgrids. *IEEE Transactions on Control Systems Technology*, 26(1):1–13, 2018.
- [4] Javier Silvente, Georgios M. Kopanos, and Antonio Espuña. A rolling horizon stochastic programming framework for the energy supply and demand management in microgrids. In *Computer Aided Chemical Engineering*, pages 2321–2326. 2015.
- [5] Huiru Zhao, Hao Lu, Bingkang Li, Xuejie Wang, Shiyong Zhang, and Yuwei Wang. Stochastic optimization of microgrid participating day-ahead market operation strategy with consideration of energy storage system and demand response. *Energies*, 13(5):1255, 2020.

A 代表日结果与题目指定表

本附录汇总四个代表日在四种年度主策略下的关键量。题目要求的五份完整工作簿与全部 10 分钟明细同时保存在支撑材料中；其表头保持题面格式。表8中的费用为当天真实执行后的现金费用，不是 0 时计划费用。

表 8 四个代表日的策略汇总

方案	日期	初始合同/kWh	最终合同/kWh	费用/元	应急/kWh
q2	03-20	74529.392	74529.392	45537.757	0.000
q2	06-21	67667.009	67667.009	41425.384	0.000
q2	09-23	70538.625	70538.625	51144.401	1056.415
q2	12-21	94119.050	94119.050	63398.423	472.787
q3	03-20	72262.710	67693.280	43102.243	0.000
q3	06-21	68371.803	64768.025	41363.804	0.000
q3	09-23	82180.247	77331.421	50781.342	0.000
q3	12-21	93441.469	93316.722	66531.379	1077.747
q4 无调账	03-20	74529.397	74529.397	46666.403	0.000
q4 无调账	06-21	67787.879	67787.879	34810.008	0.000
q4 无调账	09-23	70617.449	70617.449	52476.718	1031.465
q4 无调账	12-21	93762.134	93762.134	74326.656	472.787

方案	日期	初始合同/kWh	最终合同/kWh	费用/元	应急/kWh
q4 有调账	03-20	72244.199	67686.030	44012.157	0.000
q4 有调账	06-21	68487.694	64892.196	34896.002	0.000
q4 有调账	09-23	82197.119	77330.673	53220.330	0.000
q4 有调账	12-21	93101.259	93043.541	78804.680	1057.393

题目指定的购电表从每个代表日分别提取 10:00–10:10、12:00–12:10、14:00–14:10、16:00–16:10、18:00–18:10 和 20:00–20:10 六个自然时槽；无调账方案只使用初始合同，有调账方案同时保留初始与最终合同。指定储能表将 144 槽聚合为 0–4、4–8、8–12、12–16、16–20、20–24 时六段，并列 0 时与 24 时 SOC。应急表只合并连续非零时槽，没有应急购电的日期标记为“无”。这些规则保证聚合结果与逐槽总量守恒。

为便于在正文外快速核对，表9给出代表日首末 SOC。全年模型没有每日循环约束。本次四个代表日中，除 12 月 21 日外，各策略均由上界出发并在 24 时回到上界；这是连续运行的输出，不是人为逐日重置。

表 9 代表日首末 SOC (kWh)

日期	指标	q2	q3	q4 无调账	q4 有调账
03-20	0 时/24 时	10800/10800	10800/10800	10800/10800	10800/10800
06-21	0 时/24 时	10800/10800	10800/10800	10800/10800	10800/10800
09-23	0 时/24 时	10800/10800	10800/10800	10800/10800	10800/10800
12-21	0 时/24 时	10800/10693.3525	10800/10693.3524	10800/10691.2112	10800/10691.2110

配套文件为 `result1.xlsx`、`result2.xlsx`、`result3.xlsx`、`result4-2.xlsx` 和 `result4-3.xlsx`。其中问题二和问题四无调账方案的初始合同与最终合同相同；问题三和问题四有调账方案的最终合同由 0、6、12、18 时最后一次有效更新共同决定。

B 核心算法源码

下面给出负责数据读取、预测、场景压缩、线性规划、实际执行、价格预测和四题单日求解的核心模块。年度驱动、实验驱动、结果表生成和绘图程序作为独立源文件随支撑材料提供，以便运行和检查。本附录源码按行自动换行，长标识符的视觉折行不改变文件内容。

```

1 """2026 CUMCM C problem: shared data, forecasting, LP, and causal simulation core."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 import os
7 from pathlib import Path
8 from typing import Any
9
10 import numpy as np
11 import pandas as pd
12 from scipy.optimize import linprog
13 from scipy.sparse import csr_matrix, lil_matrix

```

```

14 os.environ.setdefault("LOKY_MAX_CPU_COUNT", "4")
15
16 from sklearn.cluster import KMeans
17
18
19 DT_HOURS = 1.0 / 6.0
20 N_SLOTS = 144
21 ETA_C = 0.90
22 ETA_D = 0.90
23 SOC_MIN = 1_200.0
24 SOC_MAX = 10_800.0
25 SOC_INITIAL = 6_000.0
26 POWER_LIMIT_KW = 5_000.0
27 ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS
28 SEED = 2026
29 DEFAULT_ALPHA = 0.80
30 DEFAULT_RISK_MODE = "unclustered"
31 ISSUES = (0, 6, 12, 18)
32
33
34 @dataclass(frozen=True)
35 class Battery:
36     capacity_kwh: float = 12000.0
37     power_kw: float = POWER_LIMIT_KW
38     eta_c: float = ETA_C
39     eta_d: float = ETA_D
40
41     @property
42     def minimum(self) -> float:
43         return 0.1 * self.capacity_kwh
44
45     @property
46     def maximum(self) -> float:
47         return 0.9 * self.capacity_kwh
48
49     @property
50     def initial(self) -> float:
51         return 0.5 * self.capacity_kwh
52
53     @property
54     def slot_limit(self) -> float:
55         return self.power_kw * DT_HOURS
56
57
58 DEFAULT_BATTERY = Battery()
59
60
61 @dataclass(frozen=True)
62 class Inputs:
63     dates: pd.DatetimeIndex
64     typical_time: list[Any]
65     typical_price: np.ndarray
66     typical_load_kw: np.ndarray
67     typical_pv_kw: np.ndarray
68     actual_load_kw: np.ndarray
69     actual_pv_kw: np.ndarray
70     variable_price: np.ndarray
71     pv_forecast_hourly: np.ndarray
72
73
74 def load_inputs(project_root: str | Path) -> Inputs:
75     root = Path(project_root)
76     attachment = root / "data" / "附件"
77

```

```

78 a1 = pd.read_excel(attachment / " 附件 1.xlsx", sheet_name=0, header=0)
79 if a1.shape != (N_SLOTS, 4):
80     raise ValueError(f" 附件 1 维度异常: {a1.shape}")
81
82 load_df = pd.read_excel(attachment / "附件2.xlsx", sheet_name="小区负载", index_col=0, header=0)
83 pv_df = pd.read_excel(attachment / "附件2.xlsx", sheet_name="光伏发电实际功率", index_col=0,
84 header=0)
85 price_df = pd.read_excel(attachment / " 附件 4.xlsx", sheet_name=0, index_col=0, header=0)
86 def clock_minute(value):
87     if hasattr(value, "hour"):
88         minute = value.hour * 60 + value.minute
89     else:
90         clock = str(value).split("+")[0].split(":")
91         minute = int(clock[0]) * 60 + int(clock[1])
92     return 1440 if minute == 0 else minute
93 expected_slots = list(range(10, 1441, 10))
94 for labels in (a1.iloc[:, 0], load_df.columns, pv_df.columns, price_df.columns):
95     if [clock_minute(v) for v in labels] != expected_slots:
96         raise ValueError(" 附件时间列不是有序的 00:10 至 24:00 端点")
97 dates = pd.DatetimeIndex(pd.to_datetime(load_df.index))
98 if load_df.shape != (365, N_SLOTS) or pv_df.shape != load_df.shape:
99     raise ValueError(" 附件 2 应为 365×144 的负荷和光伏矩阵")
100 if price_df.shape != load_df.shape:
101     raise ValueError(" 附件 4 应与附件 2 严格同形")
102 if not dates.equals(pd.DatetimeIndex(pd.to_datetime(pv_df.index))):
103     raise ValueError(" 附件 2 两个工作表日期不一致")
104 if not dates.equals(pd.DatetimeIndex(pd.to_datetime(price_df.index))):
105     raise ValueError(" 附件 4 日期与附件 2 不一致")
106
107 a3 = pd.read_excel(attachment / " 附件 3.xlsx", sheet_name=0, header=0)
108 if a3.shape != (1460, 26) or list(a3.columns[2:]) != [f" 预报 {h} 小时" for h in range(1, 25)]:
109     raise ValueError(" 附件 3 维度或提前期表头异常")
110 forecast_dates = pd.to_datetime(a3.iloc[:, 0].replace("", np.nan).ffill())
111 forecast = np.full((len(dates), len(ISSUES), 24), np.nan, dtype=float)
112 date_pos = {d.normalize(): i for i, d in enumerate(dates)}
113 issue_pos = {h: i for i, h in enumerate(ISSUES)}
114 for row_index, row in enumerate(a3.itertuples(index=False, name=None)):
115     d = pd.Timestamp(forecast_dates.iloc[row_index]).normalize()
116     issue_value = row[1]
117     issue_hour = int(issue_value.hour) if hasattr(issue_value, "hour") else
118     int(str(issue_value).split(":")[0])
119     if d not in date_pos or issue_hour not in issue_pos:
120         raise ValueError(" 附件 3 含未定义日期或发布时刻")
121     target = forecast[date_pos[d], issue_pos[issue_hour]]
122     if np.isfinite(target).any():
123         raise ValueError(" 附件 3 日期与发布时刻重复")
124     target[:] = np.asarray(row[2:26], dtype=float)
125 if np.isnan(forecast).any():
126     raise ValueError(" 附件 3 存在未能映射的日期、发布时刻或预测值")
127
128 result = Inputs(
129     dates=dates,
130     typical_time=a1.iloc[:, 0].tolist(),
131     typical_price=a1.iloc[:, 1].to_numpy(dtype=float),
132     typical_load_kw=a1.iloc[:, 2].to_numpy(dtype=float),
133     typical_pv_kw=a1.iloc[:, 3].to_numpy(dtype=float),
134     actual_load_kw=load_df.to_numpy(dtype=float),
135     actual_pv_kw=pv_df.to_numpy(dtype=float),
136     variable_price=price_df.to_numpy(dtype=float),
137     pv_forecast_hourly=forecast,
138 )
139 _validate_inputs(result)
140 return result

```

```

139
140
141 def _validate_inputs(data: Inputs) -> None:
142     arrays = (
143         data.typical_price,
144         data.typical_load_kw,
145         data.typical_pv_kw,
146         data.actual_load_kw,
147         data.actual_pv_kw,
148         data.variable_price,
149         data.pv_forecast_hourly,
150     )
151     if any(not np.isfinite(a).all() for a in arrays):
152         raise ValueError(" 输入包含 NaN 或无穷值")
153     if any((a < 0).any() for a in arrays):
154         raise ValueError(" 输入包含负的电价、负荷或光伏值")
155     expected = pd.date_range("2025-01-01", "2025-12-31", freq="D")
156     if not data.dates.equals(expected):
157         raise ValueError(" 附件日期并非 2025 完整连续自然年")
158
159
160 def build_analog_forecasts(
161     actual_kw: np.ndarray,
162     typical_kw: np.ndarray,
163     *,
164     weekday_weight: bool,
165     max_history: int = 90,
166     decay_days: float = 60.0,
167 ) -> tuple[np.ndarray, np.ndarray]:
168     """Strict rolling-origin analog forecast and its realized residuals."""
169     n_days, n_slots = actual_kw.shape
170     pred = np.empty_like(actual_kw, dtype=float)
171     for d in range(n_days):
172         start = max(0, d - max_history)
173         idx = np.arange(start, d)
174         if idx.size == 0:
175             pred[d] = typical_kw
176             continue
177         ages = d - idx
178         weights = np.exp(-ages / decay_days)
179         if weekday_weight:
180             weights *= np.where(idx % 7 == d % 7, 2.0, 1.0)
181         hist_mean = np.average(actual_kw[idx], axis=0, weights=weights)
182         shrink = min(1.0, idx.size / 21.0)
183         pred[d] = shrink * hist_mean + (1.0 - shrink) * typical_kw
184     pred = np.maximum(pred, 0.0)
185     return pred, actual_kw - pred
186
187
188 def _cluster_residuals(
189     load_residual_kw: np.ndarray,
190     pv_residual_kw: np.ndarray,
191     *,
192     max_scenarios: int = 9,
193 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
194     n = load_residual_kw.shape[0]
195     if n == 0:
196         zeros = np.zeros((1, load_residual_kw.shape[1]), dtype=float)
197         return zeros, zeros.copy(), np.ones(1)
198     raw = np.concatenate([load_residual_kw, pv_residual_kw], axis=1)
199     scale = np.std(raw, axis=0)
200     scale[scale < 1e-9] = 1.0
201     k = min(max_scenarios, n, np.unique(raw / scale, axis=0).shape[0])
202     if k == 1:

```

```

203         labels = np.zeros(n, dtype=int)
204     else:
205         labels = KMeans(n_clusters=k, random_state=SEED, n_init=10).fit_predict(raw / scale)
206     nonempty = np.unique(labels)
207     centers_l = np.stack([load_residual_kw[labels == i].mean(axis=0) for i in nonempty])
208     centers_v = np.stack([pv_residual_kw[labels == i].mean(axis=0) for i in nonempty])
209     weights = np.asarray([(labels == i).mean() for i in nonempty], dtype=float)
210     return centers_l, centers_v, weights
211
212
213 def _residual_scenarios(
214     load_residual_kw: np.ndarray,
215     pv_residual_kw: np.ndarray,
216     *,
217     risk_mode: str,
218     max_scenarios: int,
219 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
220     """Return either K-means representatives or all historical residual paths."""
221     load_residual_kw = np.asarray(load_residual_kw, dtype=float)
222     pv_residual_kw = np.asarray(pv_residual_kw, dtype=float)
223     if load_residual_kw.ndim != 2 or pv_residual_kw.shape != load_residual_kw.shape:
224         raise ValueError(" 负荷与光伏残差必须是同形二维矩阵")
225     if risk_mode == "clustered":
226         return _cluster_residuais(
227             load_residual_kw, pv_residual_kw, max_scenarios=max_scenarios
228         )
229     if risk_mode != "unclustered":
230         raise ValueError("risk_mode 必须是'clustered'或'unclustered'")
231     n = load_residual_kw.shape[0]
232     if n == 0:
233         zeros = np.zeros((1, load_residual_kw.shape[1]), dtype=float)
234         return zeros, zeros.copy(), np.ones(1)
235     return load_residual_kw.copy(), pv_residual_kw.copy(), np.full(n, 1.0 / n)
236
237
238 def weighted_quantile_columns(values: np.ndarray, weights: np.ndarray, quantile: float) ->
    np.ndarray:
239     if values.ndim != 2 or values.shape[0] != weights.size:
240         raise ValueError(" 分位数输入维度不一致")
241     if not 0 <= quantile <= 1 or not np.isfinite(values).all() or not np.isfinite(weights).all() or
    np.any(weights < 0) or weights.sum() <= 0:
242         raise ValueError(" 分位数或概率非法")
243     result = np.empty(values.shape[1], dtype=float)
244     for t in range(values.shape[1]):
245         order = np.argsort(values[:, t], kind="mergesort")
246         v = values[order, t]
247         w = weights[order]
248         cdf = np.cumsum(w) / np.sum(w)
249         result[t] = v[min(np.searchsorted(cdf, quantile, side="left"), len(v) - 1)]
250     return result
251
252
253 def safe_analog_trajectory(
254     day: int,
255     load_point_kw: np.ndarray,
256     pv_point_kw: np.ndarray,
257     load_residual_kw: np.ndarray,
258     pv_residual_kw: np.ndarray,
259     *,
260     alpha: float = DEFAULT_ALPHA,
261     history_days: int = 60,
262     max_scenarios: int = 9,
263     risk_mode: str = DEFAULT_RISK_MODE,

```

```

264 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
265     start = max(0, day - history_days)
266     residual_l = load_residual_kw[start:day]
267     residual_v = pv_residual_kw[start:day]
268     c_l, c_v, weights = _residual_scenarios(
269         residual_l,
270         residual_v,
271         risk_mode=risk_mode,
272         max_scenarios=max_scenarios,
273     )
274     l_scen = np.maximum(0.0, load_point_kw[None, :] + c_l)
275     v_scen = np.maximum(0.0, pv_point_kw[None, :] + c_v)
276     net_q = weighted_quantile_columns(l_scen - v_scen, weights, alpha)
277     pv_low = weighted_quantile_columns(v_scen, weights, 1.0 - alpha)
278     safe_pv = np.maximum.reduce([pv_low, -net_q, np.zeros_like(net_q)])
279     safe_load = safe_pv + net_q
280     return safe_load, safe_pv, {
281         "alpha": alpha,
282         "history_start": start,
283         "history_end_exclusive": day,
284         "risk_mode": risk_mode,
285         "residual_count": int(residual_l.shape[0]),
286         "scenario_count": int(weights.size),
287         "scenario_weights": weights.tolist(),
288     }
289
290
291 def _block_slices(n: int, adjusted: bool) -> dict[str, slice]:
292     pos = 0
293     blocks: dict[str, slice] = {}
294     for name, size in (("grid", n), ("charge", n), ("discharge", n), ("soc", n + 1), ("pv_used",
295         n)):
296         blocks[name] = slice(pos, pos + size)
297         pos += size
298     if adjusted:
299         blocks["up"] = slice(pos, pos + n)
300         pos += n
301         blocks["down"] = slice(pos, pos + n)
302         pos += n
303     blocks["all"] = slice(0, pos)
304     return blocks
305
306 def solve_schedule(
307     load_kw: np.ndarray,
308     pv_kw: np.ndarray,
309     price: np.ndarray,
310     soc0: float,
311     *,
312     terminal_cycle: bool = False,
313     terminal_value: float = 0.0,
314     base_plan_kwh: np.ndarray | None = None,
315     battery: Battery = DEFAULT_BATTERY,
316 ) -> dict[str, Any]:
317     """Solve a deterministic LP for an initial or adjusted contract trajectory."""
318     load_kw = np.asarray(load_kw, dtype=float)
319     pv_kw = np.asarray(pv_kw, dtype=float)
320     price = np.asarray(price, dtype=float)
321     n = load_kw.size
322     if pv_kw.shape != (n,) or price.shape != (n,):
323         raise ValueError(" 负荷、光伏、电价长度不一致")
324     if not (battery.minimum - 1e-9 <= soc0 <= battery.maximum + 1e-9):
325         raise ValueError(f" 初始 SOC 越界: {soc0}")
326     adjusted = base_plan_kwh is not None

```

```

327     if adjusted:
328         base_plan_kwh = np.asarray(base_plan_kwh, dtype=float)
329         if base_plan_kwh.shape != (n,):
330             raise ValueError(" 原计划与调整时域长度不一致")
331     b = _block_slices(n, adjusted)
332     m = b["all"].stop
333     c = np.zeros(m, dtype=float)
334     if adjusted:
335         c[b["up"]] = 1.5 * price
336         c[b["down"]] = -0.5 * price
337         constant_cost = float(np.dot(price, base_plan_kwh))
338     else:
339         c[b["grid"]] = price
340         constant_cost = 0.0
341     c[b["soc"].stop - 1] = -float(terminal_value)
342
343     rows = 2 * n + (n if adjusted else 0)
344     aeq = lil_matrix((rows, m), dtype=float)
345     beq = np.zeros(rows, dtype=float)
346     load_energy = load_kw * DT_HOURS
347     pv_energy = pv_kw * DT_HOURS
348     for t in range(n):
349         aeq[t, b["grid"].start + t] = 1.0
350         aeq[t, b["charge"].start + t] = -1.0
351         aeq[t, b["discharge"].start + t] = 1.0
352         aeq[t, b["pv_used"].start + t] = 1.0
353         beq[t] = load_energy[t]
354
355         row = n + t
356         aeq[row, b["soc"].start + t] = -1.0
357         aeq[row, b["soc"].start + t + 1] = 1.0
358         aeq[row, b["charge"].start + t] = -battery.eta_c
359         aeq[row, b["discharge"].start + t] = 1.0 / battery.eta_d
360         if adjusted:
361             row = 2 * n + t
362             aeq[row, b["grid"].start + t] = 1.0
363             aeq[row, b["up"].start + t] = -1.0
364             aeq[row, b["down"].start + t] = 1.0
365             beq[row] = base_plan_kwh[t]
366
367     bounds: list[tuple[float | None, float | None]] = []
368     bounds.extend([(0.0, None)] * n)
369     bounds.extend([(0.0, battery.slot_limit)] * n)
370     bounds.extend([(0.0, battery.slot_limit)] * n)
371     soc_bounds = [(battery.minimum, battery.maximum)] * (n + 1)
372     soc_bounds[0] = (soc0, soc0)
373     if terminal_cycle:
374         soc_bounds[-1] = (soc0, soc0)
375     bounds.extend(soc_bounds)
376     bounds.extend([(0.0, float(v)) for v in pv_energy])
377     if adjusted:
378         bounds.extend([(0.0, None)] * n)
379         bounds.extend([(0.0, float(v)) for v in base_plan_kwh])
380
381     res = linprog(c, A_eq=aeq.tocsr(), b_eq=beq, bounds=bounds, method="highs")
382     if not res.success:
383         raise RuntimeError(f"HiGHS 求解失败: status={res.status}, {res.message}")
384
385     economic_optimum = float(res.fun)
386     tie = np.zeros(m, dtype=float)
387     tie[b["charge"]] = 1.0
388     tie[b["discharge"]] = 1.0
389     aub = csr_matrix(c.reshape(1, -1))
390     tol = 1e-8 * max(1.0, abs(economic_optimum))

```

```

391     res2 = linprog(
392         tie,
393         A_ub=aub,
394         b_ub=np.asarray([economic_optimum + tol]),
395         A_eq=aeq.tocsr(),
396         b_eq=beq,
397         bounds=bounds,
398         method="highs",
399     )
400     if not res2.success:
401         raise RuntimeError(f" 吞吐次级 LP 失败: {res2.status}, {res2.message}")
402     x = res2.x
403     balance_residual = aeq.tocsr() @ x - beq
404     result = {
405         "grid_kwh": x[b["grid"]].copy(),
406         "charge_kwh": x[b["charge"]].copy(),
407         "discharge_kwh": x[b["discharge"]].copy(),
408         "soc_kwh": x[b["soc"]].copy(),
409         "pv_used_kwh": x[b["pv_used"]].copy(),
410         "solver_objective": float(np.dot(c, x) + constant_cost + terminal_value * battery.minimum),
411         "cash_contract_cost": float(
412             np.dot(price, x[b["grid"]])
413             if not adjusted
414             else constant_cost
415             + np.dot(1.5 * price, x[b["up"]])
416             - np.dot(0.5 * price, x[b["down"]])
417         ),
418         "terminal_value_credit": float(terminal_value * (x[b["soc"]].stop - 1) - battery.minimum),
419         "max_eq_residual": float(np.max(np.abs(balance_residual))),
420         "simultaneous_flow_slots": int(np.count_nonzero(np.minimum(x[b["charge"]],
421             x[b["discharge"]]) > 1e-5)),
422         "simultaneous_flow_max": float(np.max(np.minimum(x[b["charge"]], x[b["discharge"]]))),
423         "status": "optimal",
424     }
425     if adjusted:
426         result["up_kwh"] = x[b["up"]].copy()
427         result["down_kwh"] = x[b["down"]].copy()
428     return result
429
430 def causal_dispatch(
431     contract_kwh: np.ndarray,
432     actual_load_kw: np.ndarray,
433     actual_pv_kw: np.ndarray,
434     price: np.ndarray,
435     soc0: float,
436     *,
437     base_plan_kwh: np.ndarray | None = None,
438     battery: Battery = DEFAULT_BATTERY,
439     emergency_multiplier: float = 5.0,
440 ) -> dict[str, Any]:
441     """Greedy feasible controller using current observation only."""
442     q = np.asarray(contract_kwh, dtype=float)
443     load_e = np.asarray(actual_load_kw, dtype=float) * DT_HOURS
444     pv_e = np.asarray(actual_pv_kw, dtype=float) * DT_HOURS
445     price = np.asarray(price, dtype=float)
446     n = q.size
447     if load_e.shape != (n,) or pv_e.shape != (n,) or price.shape != (n,):
448         raise ValueError(" 因果控制输入长度不一致")
449     charge = np.zeros(n)
450     discharge = np.zeros(n)
451     emergency = np.zeros(n)
452     unused = np.zeros(n)
453     curtailed = np.zeros(n)

```

```

454 pv_used = np.zeros(n)
455 soc = np.empty(n + 1)
456 soc[0] = soc0
457 for t in range(n):
458     available = q[t] + pv_e[t] - load_e[t]
459     if available >= 0.0:
460         charge[t] = min(available, battery.slot_limit, max(0.0, (battery.maximum - soc[t]) /
461             battery.eta_c))
462         surplus = max(0.0, available - charge[t])
463         unused[t] = min(q[t], surplus)
464         curtailed[t] = max(0.0, surplus - unused[t])
465         pv_used[t] = pv_e[t] - curtailed[t]
466         soc[t + 1] = soc[t] + battery.eta_c * charge[t]
467     else:
468         need = -available
469         discharge[t] = min(need, battery.slot_limit, max(0.0, (soc[t] - battery.minimum) *
470             battery.eta_d))
471         emergency[t] = max(0.0, need - discharge[t])
472         pv_used[t] = pv_e[t]
473         soc[t + 1] = soc[t] - discharge[t] / battery.eta_d
474     residual = q - unused + pv_used + discharge + emergency - load_e - charge
475     if base_plan_kwh is None:
476         settlement = price * q
477         alt_settlement = settlement.copy()
478     else:
479         p = np.asarray(base_plan_kwh, dtype=float)
480         if p.shape != (n,):
481             raise ValueError(" 原计划与最终合同长度不一致")
482         settlement = price * np.minimum(p, q) + 1.5 * price * np.maximum(q - p, 0.0) + 0.5 * price
483         * np.maximum(p - q, 0.0)
484         alt_settlement = price * p + 1.5 * price * np.maximum(q - p, 0.0) + 0.5 * price *
485         np.maximum(p - q, 0.0)
486     emergency_cost = emergency_multiplier * price * emergency
487     return {
488         "charge_kwh": charge,
489         "discharge_kwh": discharge,
490         "soc_kwh": soc,
491         "emergency_kwh": emergency,
492         "unused_contract_kwh": unused,
493         "pv_curtailed_kwh": curtailed,
494         "pv_used_kwh": pv_used,
495         "settlement_cost": settlement,
496         "alternative_settlement_cost": alt_settlement,
497         "emergency_cost": emergency_cost,
498         "cash_cost": settlement + emergency_cost,
499         "alternative_cash_cost": alt_settlement + emergency_cost,
500         "max_balance_residual": float(np.max(np.abs(residual))),
501         "simultaneous_flow_slots": int(np.count_nonzero(np.minimum(charge, discharge) > 1e-5)),
502         "simultaneous_flow_max": float(np.max(np.minimum(charge, discharge))),
503     }
504
505 def terminal_value(price: np.ndarray, battery: Battery = DEFAULT_BATTERY) -> float:
506     return float(battery.eta_d * np.median(np.asarray(price, dtype=float)))
507
508 def predict_price(data: Inputs, day: int, issue_hour: int = 0) -> np.ndarray:
509     """Only historical days and the already-realized prefix are readable."""
510     base = np.median(data.variable_price[max(0, day - 30):day], axis=0) if day else
511     data.typical_price.copy()
512     start = issue_hour * 6
513     if start:
514         observed = slice(max(0, start - 18), start)

```

```

512         bias = float(np.mean(data.variable_price[day, observed] - base[observed]))
513         tau = np.arange(1, N_SLOTS - start + 1) / 6.0
514         return np.maximum(0.0001, base[start:] + bias * np.exp(-tau / 3.0))
515     return np.maximum(0.0001, base)
516
517
518 def run_q1(data: Inputs, battery: Battery = DEFAULT_BATTERY) -> dict[str, Any]:
519     return solve_schedule(
520         data.typical_load_kw,
521         data.typical_pv_kw,
522         data.typical_price,
523         battery.initial,
524         terminal_cycle=True,
525         battery=battery,
526     )
527
528
529 def run_q2_day(
530     data: Inputs,
531     day: int,
532     soc0: float,
533     load_point: np.ndarray,
534     pv_point: np.ndarray,
535     load_residual: np.ndarray,
536     pv_residual: np.ndarray,
537     *,
538     variable_price: bool,
539     alpha: float = DEFAULT_ALPHA,
540     safe_pair: tuple[np.ndarray, np.ndarray, dict[str, Any]] | None = None,
541     battery: Battery = DEFAULT_BATTERY,
542     emergency_multiplier: float = 5.0,
543     history_days: int = 60,
544     max_scenarios: int = 9,
545     risk_mode: str = DEFAULT_RISK_MODE,
546 ) -> dict[str, Any]:
547     price = predict_price(data, day) if variable_price else data.typical_price
548     settlement_price = data.variable_price[day] if variable_price else data.typical_price
549     if safe_pair is None:
550         safe_l, safe_v, meta = safe_analog_trajectory(
551             day, load_point[day], pv_point[day], load_residual, pv_residual,
552             alpha=alpha, history_days=history_days,
553             max_scenarios=max_scenarios, risk_mode=risk_mode,
554         )
555     else:
556         safe_l, safe_v, meta = safe_pair
557     plan = solve_schedule(
558         safe_l,
559         safe_v,
560         price,
561         soc0,
562         terminal_value=terminal_value(price, battery),
563         battery=battery,
564     )
565     actual = causal_dispatch(
566         plan["grid_kwh"], data.actual_load_kw[day], data.actual_pv_kw[day], settlement_price, soc0,
567         battery=battery, emergency_multiplier=emergency_multiplier
568     )
569     return {"plan": plan, "actual": actual, "forecast_meta": meta, "safe_load_kw": safe_l,
570             "safe_pv_kw": safe_v, "planning_price": price}
571
572
573 def official_pv_residuals(data: Inputs) -> np.ndarray:
574     """Actual power at each forecasted clock hour minus that point forecast."""

```

```

573     n_days = len(data.dates)
574     flat = data.actual_pv_kw.reshape(-1)
575     residual = np.full_like(data.pv_forecast_hourly, np.nan, dtype=float)
576     for d in range(n_days):
577         for i, issue in enumerate(ISSUES):
578             start = d * N_SLOTS + issue * 6
579             for h in range(24):
580                 # Column h is the point forecast at issue+(h+1) hours. The
581                 # 10-minute source rows are interval endpoints, hence -1.
582                 endpoint = start + (h + 1) * 6 - 1
583                 if endpoint < flat.size:
584                     residual[d, i, h] = flat[endpoint] - data.pv_forecast_hourly[d, i, h]
585     return residual
586
587
588 def interpolate_hourly_points(anchor: float, hourly_points: np.ndarray, n_slots: int) ->
    np.ndarray:
589     """Linearly interpolate clock-hour point values onto 10-minute endpoints."""
590     values = np.concatenate([float(anchor)], np.asarray(hourly_points, dtype=float)))
591     x_hour = np.arange(values.size, dtype=float)
592     x_slot = np.arange(1, n_slots + 1, dtype=float) / 6.0
593     return np.interp(x_slot, x_hour, values)
594
595
596 def safe_update_trajectory(
597     data: Inputs,
598     day: int,
599     issue_index: int,
600     load_point_day: np.ndarray,
601     load_residual: np.ndarray,
602     pv_issue_residual: np.ndarray,
603     *,
604     alpha: float = DEFAULT_ALPHA,
605     history_days: int = 60,
606     max_scenarios: int = 9,
607     risk_mode: str = DEFAULT_RISK_MODE,
608 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
609     issue = ISSUES[issue_index]
610     start_slot = issue * 6
611     n = N_SLOTS - start_slot
612     hours = int(np.ceil(n / 6))
613     raw = data.pv_forecast_hourly[day, issue_index, :hours]
614     hist_start = max(0, day - history_days)
615     hist = pv_issue_residual[hist_start:day, issue_index, :hours]
616     valid_rows = np.all(np.isfinite(hist), axis=1) if hist.size else np.zeros(0, dtype=bool)
617     hist = hist[valid_rows]
618     bias = np.nanmean(hist, axis=0) if hist.shape[0] else np.zeros(hours)
619     if issue > 0:
620         anchor = float(data.actual_pv_kw[day, start_slot - 1])
621     elif day > 0:
622         anchor = float(data.actual_pv_kw[day - 1, -1])
623     else:
624         anchor = 0.0
625     pv_point = interpolate_hourly_points(anchor, np.maximum(0.0, raw + bias), n)
626     l_point = load_point_day[start_slot:]
627
628     hist_ids = np.arange(hist_start, day)[valid_rows]
629     if hist_ids.size:
630         r_l = load_residual[hist_ids, start_slot:]
631         centered_hourly = hist - bias[None, :]
632         r_v = np.vstack(
633             [interpolate_hourly_points(0.0, row, n) for row in centered_hourly]
634         )
635     else:

```

```

636         r_l = np.empty((0, n))
637         r_v = np.empty((0, n))
638     c_l, c_v, weights = _residual_scenarios(
639         r_l, r_v, risk_mode=risk_mode, max_scenarios=max_scenarios
640     )
641     l_scen = np.maximum(0.0, l_point[None, :] + c_l)
642     v_scen = np.maximum(0.0, pv_point[None, :] + c_v)
643     net_q = weighted_quantile_columns(l_scen - v_scen, weights, alpha)
644     pv_low = weighted_quantile_columns(v_scen, weights, 1.0 - alpha)
645     safe_v = np.maximum.reduce([pv_low, -net_q, np.zeros_like(net_q)])
646     safe_l = safe_v + net_q
647     return safe_l, safe_v, {
648         "decision_day": int(day),
649         "issue_hour": int(issue),
650         "history_start": int(hist_start),
651         "history_end_exclusive": int(day),
652         "risk_mode": risk_mode,
653         "residual_count": int(hist_ids.size),
654         "scenario_count": int(weights.size),
655         "alpha": float(alpha),
656         "scenario_weights": weights.tolist(),
657         "pv_point_kw": pv_point.tolist(),
658         "anchor_kw": anchor,
659         "bias_kw": bias.tolist(),
660     }
661
662
663 def nowcast_trajectory(
664     data: Inputs, day: int, event: int,
665     source: tuple[np.ndarray, np.ndarray, dict[str, Any]],
666 ) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
667     """Synthetic nowcast based on the latest available official issue only."""
668     safe_l, safe_v, meta = source
669     issue = int(meta["issue_hour"])
670     offset = (event - issue) * 6
671     if offset < 0:
672         raise ValueError(" 不能使用未来发布的预报")
673     if not offset:
674         return source
675     point = np.asarray(meta["pv_point_kw"])
676     first = max(0, offset - 18)
677     realized = data.actual_pv_kw[day, issue * 6 + first:event * 6]
678     bias = float(np.mean(realized - point[first:offset])) if realized.size else 0.0
679     tau = np.arange(1, N_SLOTS - event * 6 + 1) / 6.0
680     revised = np.maximum(0.0, safe_v[offset:] + bias * np.exp(-tau / 3.0))
681     return safe_l[offset:], revised, {
682         **{k: v for k, v in meta.items() if k not in ("pv_point_kw", "bias_kw")},
683         "decision_hour": event, "synthetic_nowcast": True,
684         "observed_end_exclusive": event * 6, "nowcast_bias_kw": bias,
685     }
686
687
688 def run_q3_day(
689     data: Inputs,
690     day: int,
691     soc0: float,
692     load_point: np.ndarray,
693     load_residual: np.ndarray,
694     pv_issue_residual: np.ndarray,
695     *,
696     variable_price: bool,
697     alpha: float = DEFAULT_ALPHA,
698     safe_updates: list[tuple[np.ndarray, np.ndarray, dict[str, Any]]] | None = None,
699     events: tuple[int, ...] = ISSUES,

```

```

700     allowed_issue_hours: tuple[int, ...] = ISSUES[1:],
701     battery: Battery = DEFAULT_BATTERY,
702     emergency_multiplier: float = 5.0,
703     history_days: int = 60,
704     max_scenarios: int = 9,
705     risk_mode: str = DEFAULT_RISK_MODE,
706 ) -> dict[str, Any]:
707     if not events or events[0] != 0 or tuple(sorted(set(events))) != events or any(h < 0 or h >= 24
or int(h) != h for h in events):
708         raise ValueError(" 更新时刻必须从 0 开始, 严格递增且在 [0,24) 内")
709     allowed_issue_hours = tuple(allowed_issue_hours)
710     if tuple(sorted(set(allowed_issue_hours))) != allowed_issue_hours or any(h not in ISSUES[1:]
for h in allowed_issue_hours):
711         raise ValueError(" 获准新发布时间必须是 {6,12,18} 的严格递增子集")
712     readable_issues = (0,) + allowed_issue_hours
713     price = predict_price(data, day) if variable_price else data.typical_price
714     settlement_price = data.variable_price[day] if variable_price else data.typical_price
715     if safe_updates is None:
716         safe_l0, safe_v0, meta0 = safe_update_trajectory(
717             data, day, 0, load_point[day], load_residual, pv_issue_residual,
718             alpha=alpha, history_days=history_days,
719             max_scenarios=max_scenarios, risk_mode=risk_mode,
720         )
721     else:
722         safe_l0, safe_v0, meta0 = safe_updates[0]
723     initial = solve_schedule(
724         safe_l0,
725         safe_v0,
726         price,
727         soc0,
728         terminal_value=terminal_value(price, battery),
729         battery=battery,
730     )
731     p0 = initial["grid_kwh"]
732     final_contract = p0.copy()
733     actual_parts: dict[str, list[np.ndarray]] = {
734         key: []
735         for key in (
736             "charge_kwh",
737             "discharge_kwh",
738             "emergency_kwh",
739             "unused_contract_kwh",
740             "pv_curtailed_kwh",
741             "pv_used_kwh",
742             "settlement_cost",
743             "alternative_settlement_cost",
744             "emergency_cost",
745             "cash_cost",
746             "alternative_cash_cost",
747         )
748     }
749     soc_trace = [float(soc0)]
750     update_meta = [{
751         **meta0,
752         "decision_hour": 0,
753         "source_issue_hour": 0,
754         "is_new_official_release": True,
755         "allowed_issue_hours": list(allowed_issue_hours),
756     }]
757     update_solutions: list[dict[str, Any]] = []
758     target_soc = np.full(N_SLOTS + 1, np.nan, dtype=float)
759     safe_net_kw = np.full(N_SLOTS, np.nan, dtype=float)
760     current_soc = float(soc0)

```

```

761
762 for i, issue in enumerate(events):
763     start = issue * 6
764     end = events[i + 1] * 6 if i + 1 < len(events) else N_SLOTS
765     if i == 0:
766         safe_l, safe_v = safe_l0, safe_v0
767         contract_remaining = p0[start:]
768         update_solution = initial
769     else:
770         source_issue = max(h for h in readable_issues if h <= issue)
771         latest_index = ISSUES.index(source_issue)
772         if safe_updates is None:
773             source = safe_update_trajectory(
774                 data, day, latest_index, load_point[day], load_residual,
775                 pv_issue_residual, alpha=alpha, history_days=history_days,
776                 max_scenarios=max_scenarios, risk_mode=risk_mode,
777             )
778         else:
779             source = safe_updates[latest_index]
780         safe_l, safe_v, meta = nowcast_trajectory(data, day, issue, source)
781         meta = {
782             **meta,
783             "decision_hour": int(issue),
784             "source_issue_hour": int(source_issue),
785             "is_new_official_release": bool(issue == source_issue),
786             "allowed_issue_hours": list(allowed_issue_hours),
787         }
788         update_meta.append(meta)
789         remaining_price = predict_price(data, day, issue) if variable_price else price[start:]
790         update_solution = solve_schedule(
791             safe_l,
792             safe_v,
793             remaining_price,
794             current_soc,
795             terminal_value=terminal_value(remaining_price, battery),
796             base_plan_kwh=p0[start:],
797             battery=battery,
798         )
799         update_solution["planning_price"] = remaining_price
800         contract_remaining = update_solution["grid_kwh"]
801         update_solutions.append(update_solution)
802         target_soc[start:end + 1] = update_solution["soc_kwh"][: end - start + 1]
803         safe_net_kw[start:end] = (safe_l - safe_v)[: end - start]
804         segment_contract = contract_remaining[: end - start]
805         final_contract[start:end] = segment_contract
806         segment = causal_dispatch(
807             segment_contract,
808             data.actual_load_kw[day, start:end],
809             data.actual_pv_kw[day, start:end],
810             settlement_price[start:end],
811             current_soc,
812             base_plan_kwh=p0[start:end],
813             battery=battery,
814             emergency_multiplier=emergency_multiplier,
815         )
816         for key in actual_parts:
817             actual_parts[key].append(segment[key])
818         soc_trace.extend(segment["soc_kwh"][1:].tolist())
819         current_soc = float(segment["soc_kwh"][-1])
820
821 actual = {key: np.concatenate(parts) for key, parts in actual_parts.items()}
822 actual["soc_kwh"] = np.asarray(soc_trace)

```

```

823     balance = final_contract - actual["unused_contract_kwh"] + actual["pv_used_kwh"] +
        actual["discharge_kwh"] + actual["emergency_kwh"] - data.actual_load_kw[day] * DT_HOURS -
        actual["charge_kwh"]
824     actual["max_balance_residual"] = float(np.max(np.abs(balance)))
825     return {
826         "initial_plan": initial,
827         "initial_plan_kwh": p0,
828         "final_contract_kwh": final_contract,
829         "actual": actual,
830         "update_meta": update_meta,
831         "update_solutions": update_solutions,
832         "target_soc_kwh": target_soc,
833         "safe_net_kw": safe_net_kw,
834         "planning_price": price,
835         "events": events,
836         "allowed_issue_hours": allowed_issue_hours,
837     }
838
839
840 def summarize_day(result: dict[str, Any], *, adjusted: bool) -> dict[str, float]:
841     actual = result["actual"]
842     plan = result["initial_plan_kwh"] if adjusted else result["plan"]["grid_kwh"]
843     contract = result["final_contract_kwh"] if adjusted else plan
844     return {
845         "plan_kwh": float(np.sum(plan)),
846         "final_contract_kwh": float(np.sum(contract)),
847         "cash_cost_yuan": float(np.sum(actual["cash_cost"])),
848         "alternative_cash_cost_yuan": float(np.sum(actual.get("alternative_cash_cost",
            actual["cash_cost"]))),
849         "emergency_kwh": float(np.sum(actual["emergency_kwh"])),
850         "unused_contract_kwh": float(np.sum(actual["unused_contract_kwh"])),
851         "pv_curtailed_kwh": float(np.sum(actual["pv_curtailed_kwh"])),
852         "charge_kwh": float(np.sum(actual["charge_kwh"])),
853         "discharge_kwh": float(np.sum(actual["discharge_kwh"])),
854         "soc_start_kwh": float(actual["soc_kwh"][0]),
855         "soc_end_kwh": float(actual["soc_kwh"][-1]),
856         "soc_min_kwh": float(np.min(actual["soc_kwh"])),
857         "soc_max_kwh": float(np.max(actual["soc_kwh"])),
858         "max_balance_residual_kwh": float(actual["max_balance_residual"]),
859     }

```

C 支撑材料与运行说明

支撑材料包括以下内容：

- (1) 核心模型与执行程序：
microgrid_core.py、**run_all.py**；
evaluate_results.py；
- (2) 54 组实验、绘图与检查程序：
run_experiments.py、**plot_all.py**；
audit_results.py、**audit_experiments.py**；
- (3) 单元测试和统一入口：
test_core.py、**test_evaluation.py**；
smoke_test.py、**reproduce.py**；

- (4) 题目原始附件、五份结果工作簿、关键 CSV 汇总、15 幅论文图、19 幅逻辑图及可编辑 SVG 图；
- (5) 运行环境说明、依赖列表、模型参数说明和《AI 工具使用详情》。

在项目根目录中使用 Python 3.14 或兼容版本，安装 NumPy、pandas、SciPy、scikit-learn、openpyxl、Matplotlib 与 Pillow 后，可依次运行主结果和实验程序。完整统一入口为：

```
python -X utf8 reproduce.py --project-root .
```

该入口从原始附件读取数据，依次运行测试、四题结果、54 组实验、工作簿生成、图表输出和格式检查。跨机器使用时，应将绘图辅助工具路径配置为本地实际位置；如只需核对已有输出，可运行各审计程序而不重复年度优化。